Conectando Aplicação com Banco de dados

Objetos para conexão

Propriedade do objeto connection:

- **Connection** – Define uma conexão com o banco

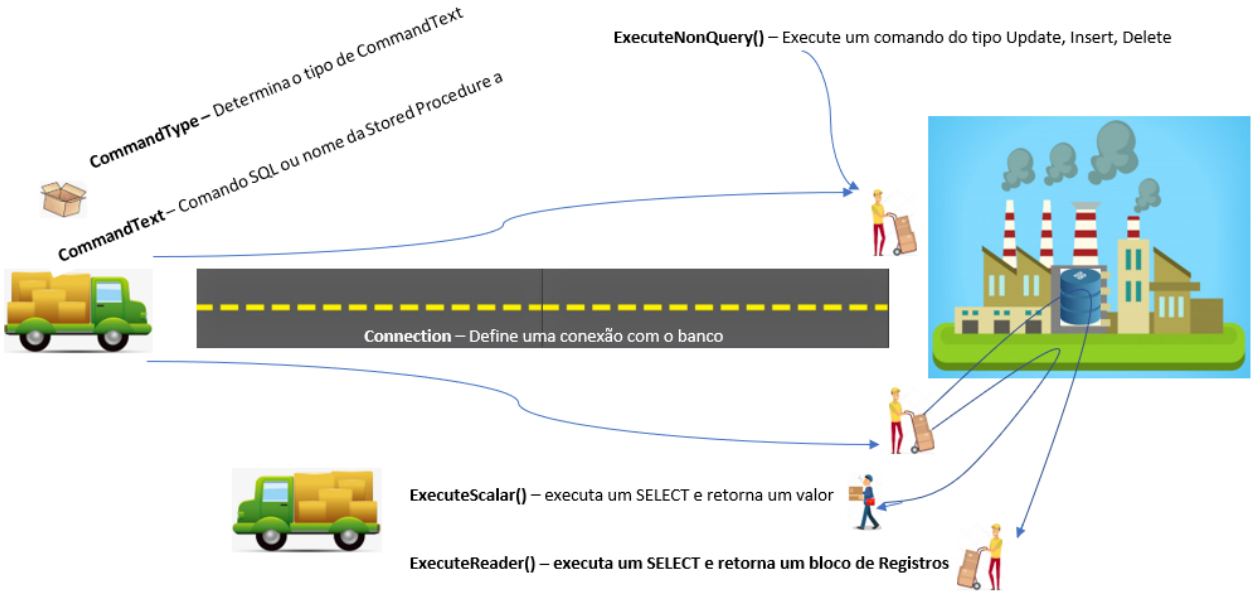
Propriedade do objeto Command:

- **CommandText** – Comando SQL ou nome da Stored Procedure a chamar
- **CommandType** – Determina o tipo de CommandText
- **Parameters** – É uma coleção de parâmetros associada a um comando

Métodos:

- **ExecuteReader()** – executa um SELECT e retorna um bloco de Registros
- **ExecuteScalar()** – executa um SELECT e retorna um valor apenas
- **ExecuteNonQuery()** – Execute um comando do tipo Update, Insert, Delete

Ilustração



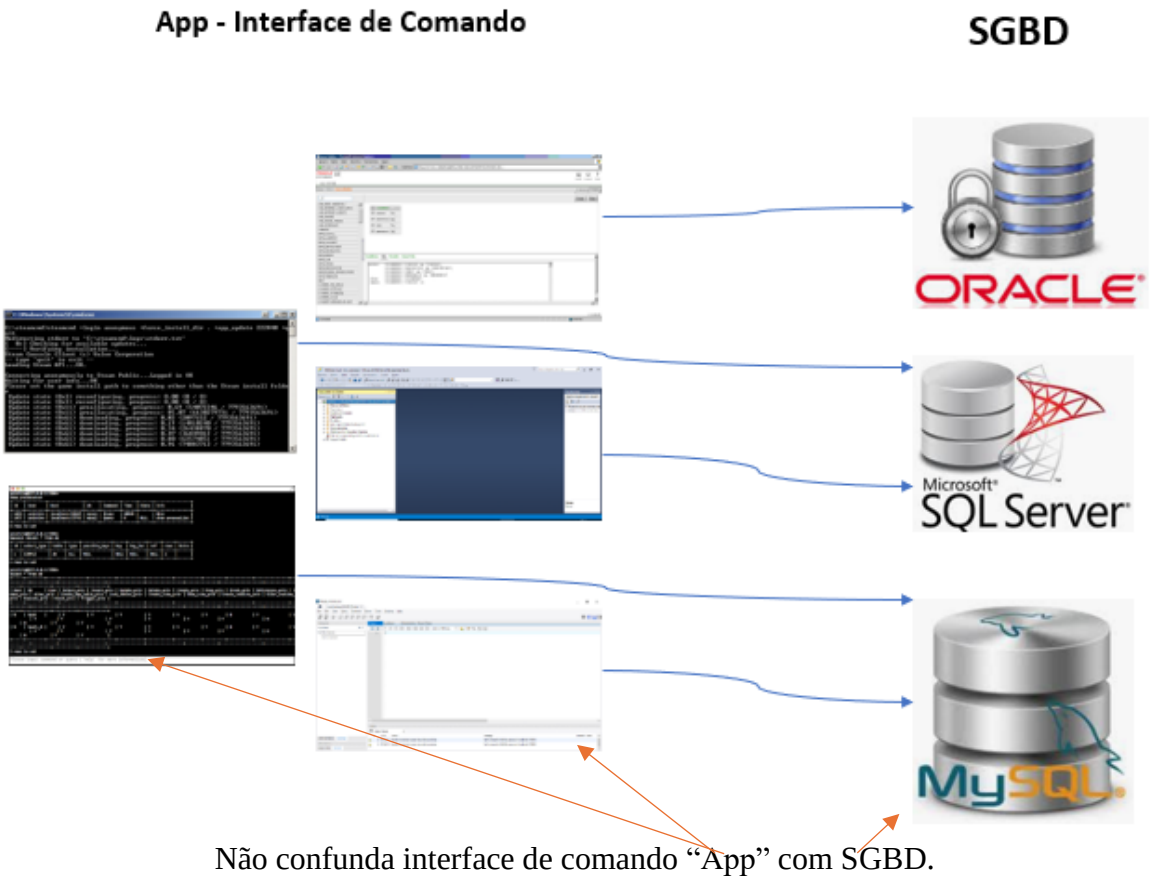
Conectar aplicações com servidores de banco de dados são todos muito parecidas, veremos a seguir exemplos e parâmetros necessários para realizar a conexão.

O objeto “Connection” é um tipo genérico, para cada SGBD terá seu objeto próprio.

Exemplo

- MySQL = MySqlConnection
- Sql Server = SqlConnection

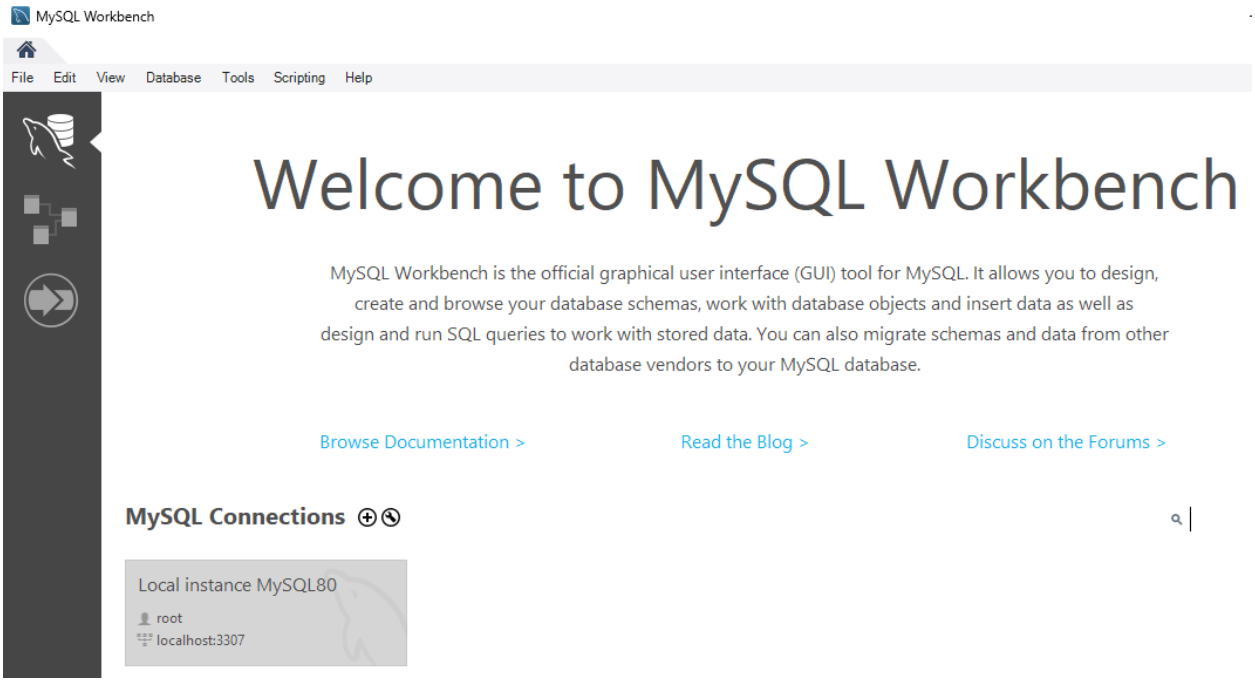
- Veja alguns exemplos de SGBDs – “Sistema de Gerenciamento de Banco de Dados”



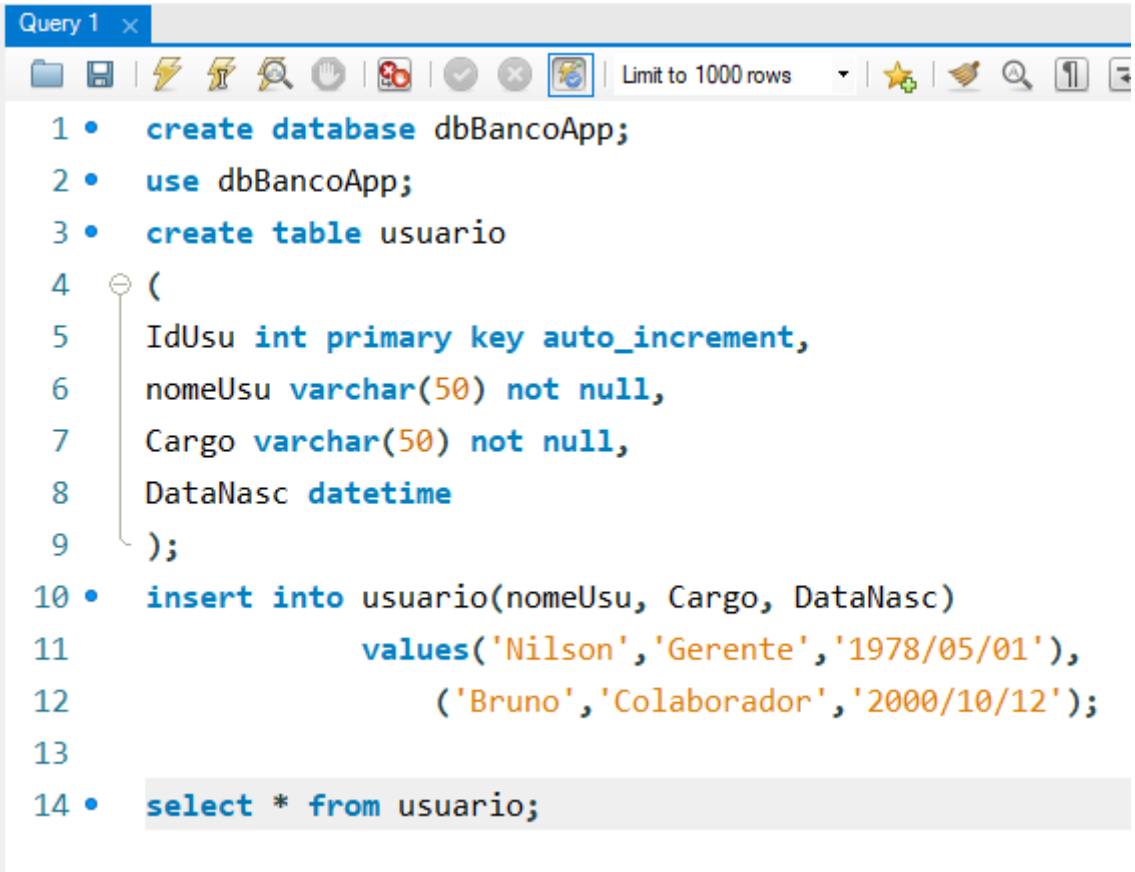
Não confunda interface de comando “App” com SGBD.

Banco de Dados

Os exemplos a seguir, serão realizados com uma base de dados no SGBD - MySql Server, para acessar o servidor utilizaremos o MySql Workbench.



Após conectar-se com o servidor, abra uma query e crie uma base de dados conforme código abaixo:



Comentário do código Sql:

-- Comando para criar uma base de dados

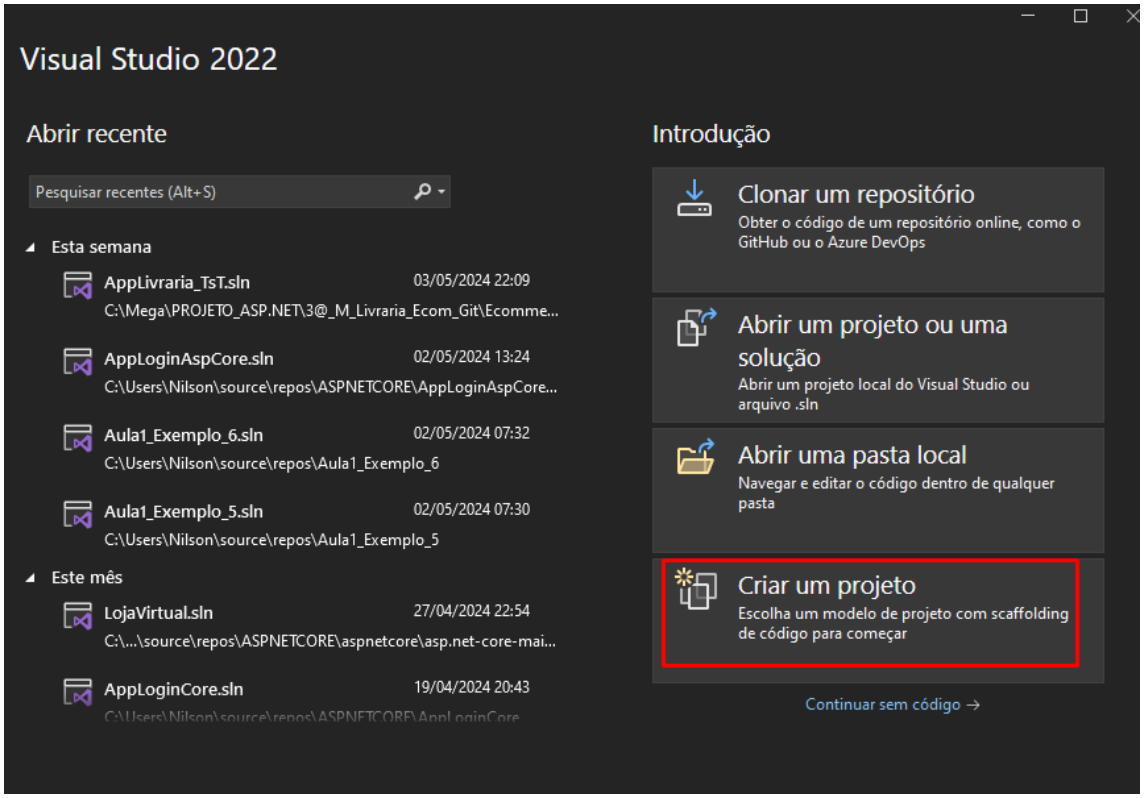
create database dbAppBanco;

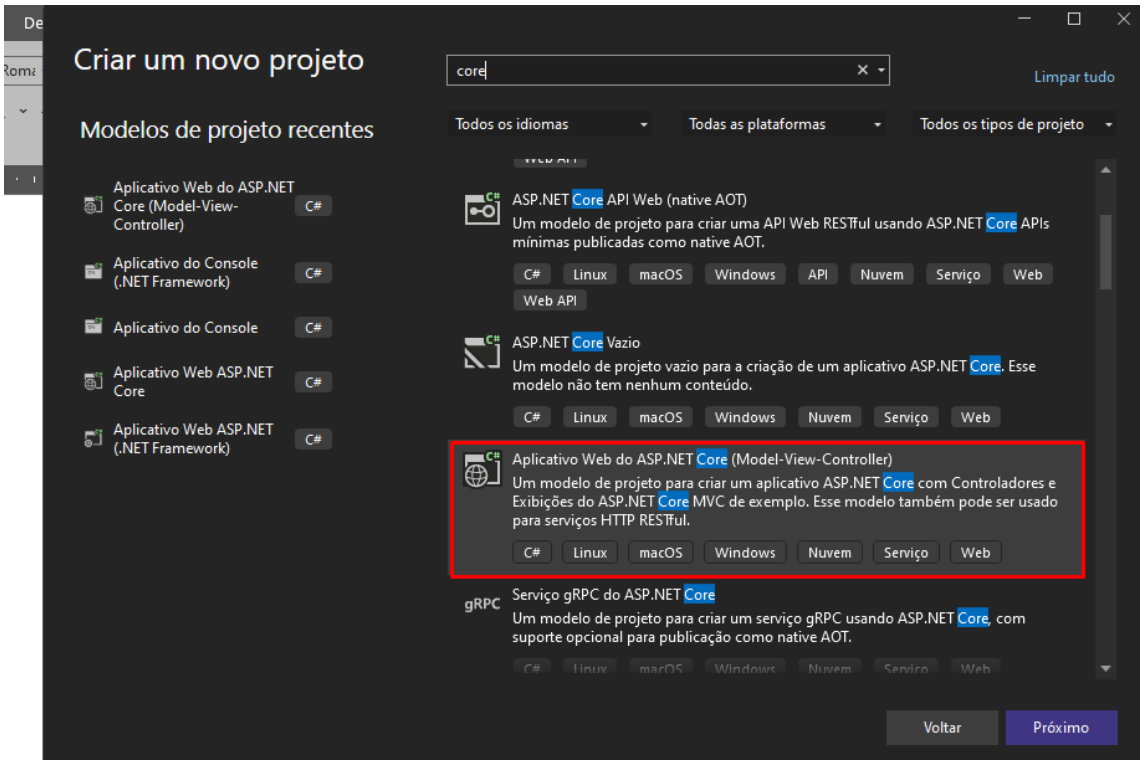
```
-- Colocando a base de dados em uso
use dbAppBanco;
-- Código para criar uma tabela
create table tbUsuario
(
IdUsu int primary key auto_increment,
NomeUsu varchar(50) not null,
Cargo varchar(50) not null,
DataNasc datetime
);
-- Inserindo registros na tabela
insert into tbUsuario(NomeUsu, Cargo, DataNasc)
                    Values(Nilson,'Gerente','1978/05/01'),
                              ('Bruno','Colaborador','2000/10/12');
-- Mostrando os registros da tabela
select * from tbUsuario;
```

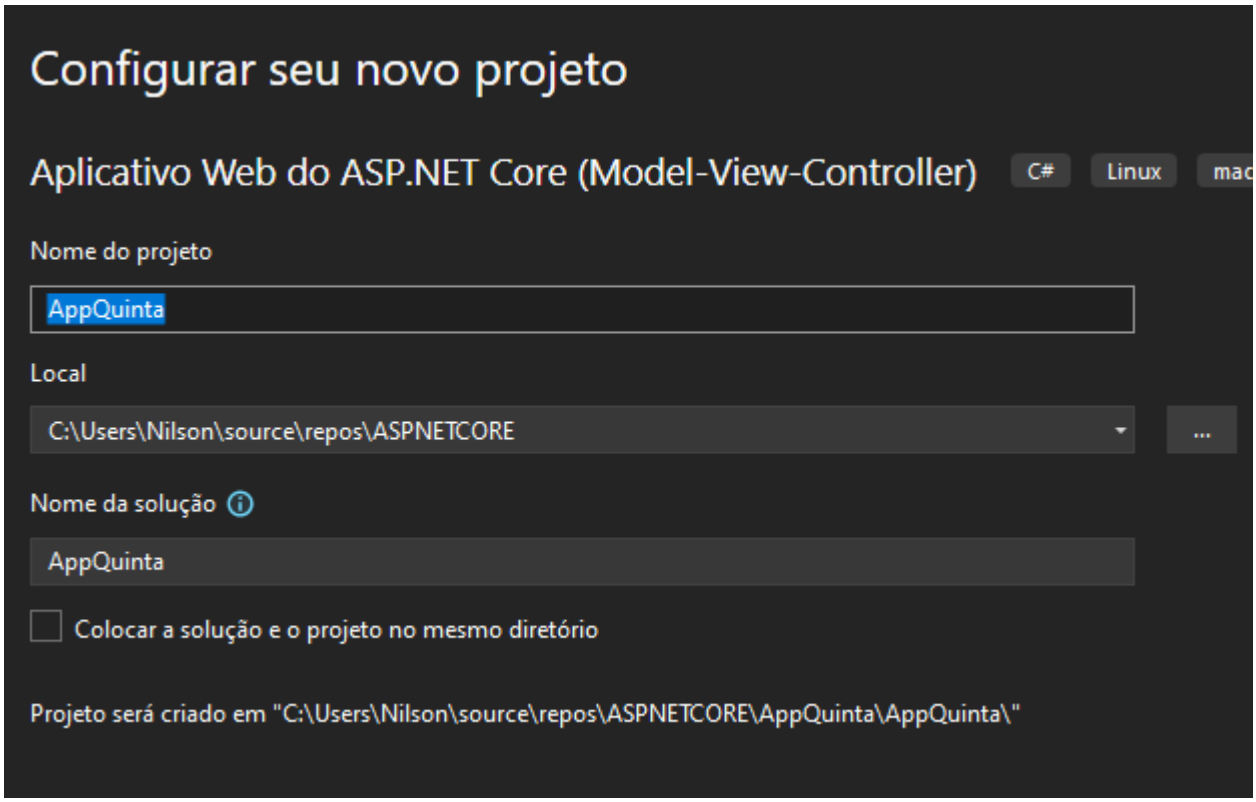
Result Grid | Filter Rows: | Edit:

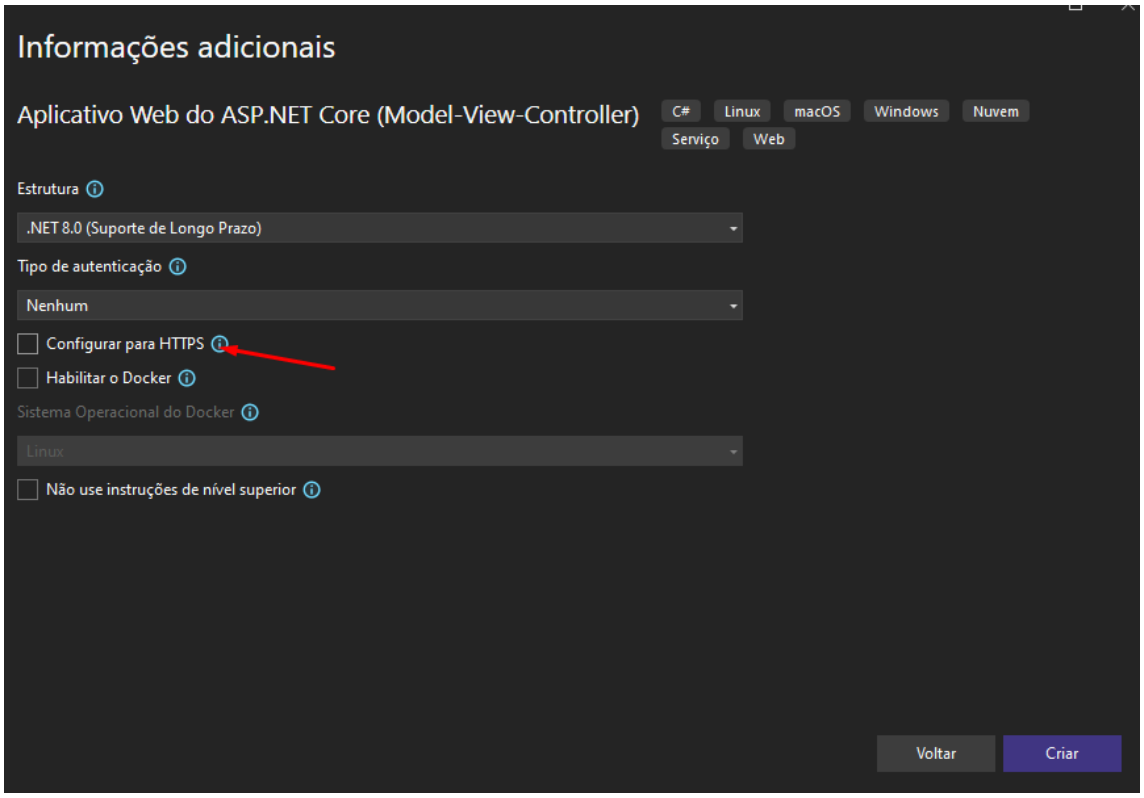
IdUsu	nomeUsu	Cargo	DataNasc
1	Nilson	Gerente	1978-05-01 00:00:00
2	Bruno	Colaborador	2000-10-12 00:00:00
NULL	NULL	NULL	NULL

Criando Aplicação Exemplo

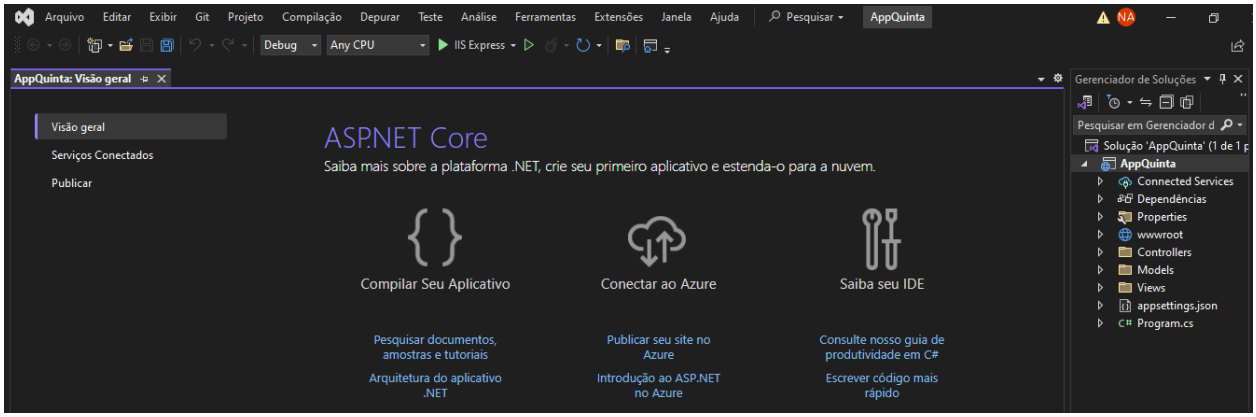



Criar um novo projeto
core
Limpar tudo
Modelos de projeto recentes
Todos os idiomas
Todas as plataformas
Todos os tipos de projeto
Aplicativo Web do ASP.NET Core (Model-View-Controller)
C#
Aplicativo do Console (.NET Framework)
C#
Aplicativo do Console
C#
Aplicativo Web ASP.NET Core
C#
Aplicativo Web ASP.NET (.NET Framework)
C#
ASP.NET Core API Web (native AOT)
Um modelo de projeto para criar uma API Web RESTful usando ASP.NET Core APIs mínimas publicadas como native AOT.
C# Linux macOS Windows API Nuvem Serviço Web Web API
ASP.NET Core Vazio
Um modelo de projeto vazio para a criação de um aplicativo ASP.NET Core. Esse modelo não tem nenhum conteúdo.
C# Linux macOS Windows Nuvem Serviço Web
Aplicativo Web do ASP.NET Core (Model-View-Controller)
Um modelo de projeto para criar um aplicativo ASP.NET Core com Controladores e Exibições do ASP.NET Core MVC de exemplo. Esse modelo também pode ser usado para serviços HTTP RESTful.
C# Linux macOS Windows Nuvem Serviço Web
gRPC
Serviço gRPC do ASP.NET Core
Um modelo de projeto para criar um serviço gRPC usando ASP.NET Core, com suporte opcional para publicação como native AOT.
Voltar
Próximo

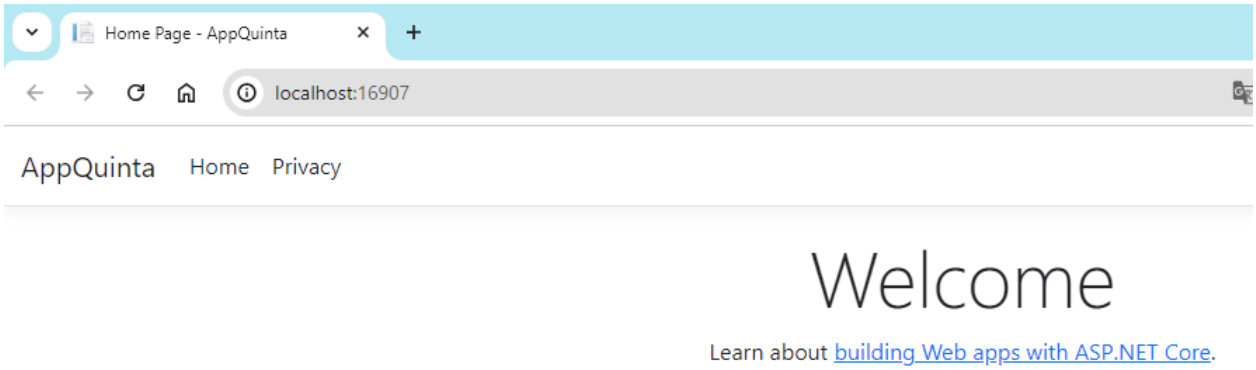

Configurar seu novo projeto
Aplicativo Web do ASP.NET Core (Model-View-Controller)
C#
Linux
Nome do projeto
AppQuinta
Local
C:\Users\Nilson\source\repos\ASPNETCORE
...
Nome da solução
AppQuinta
Colocar a solução e o projeto no mesmo diretório
Projeto será criado em "C:\Users\Nilson\source\repos\ASPNETCORE\AppQuinta\AppQuinta\"



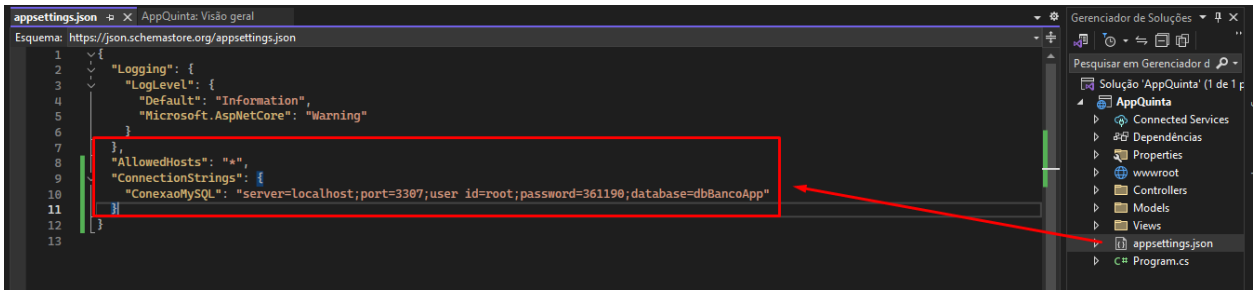
Após criar execute a aplicação



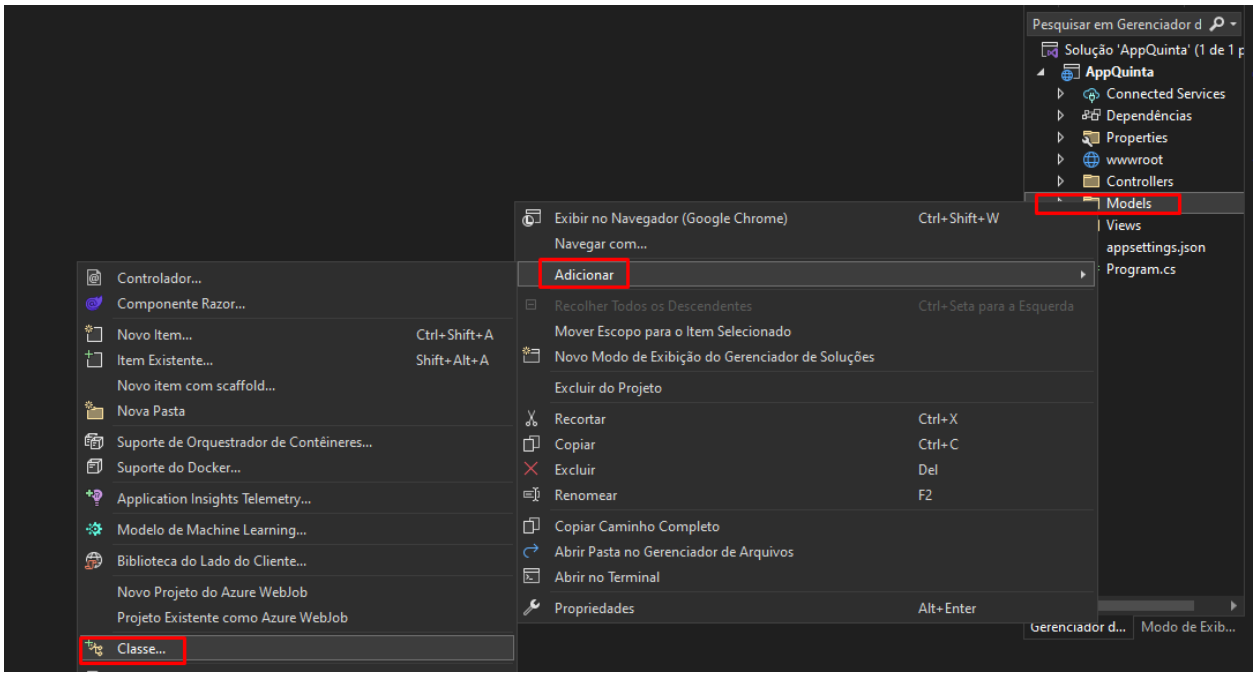
Pronto

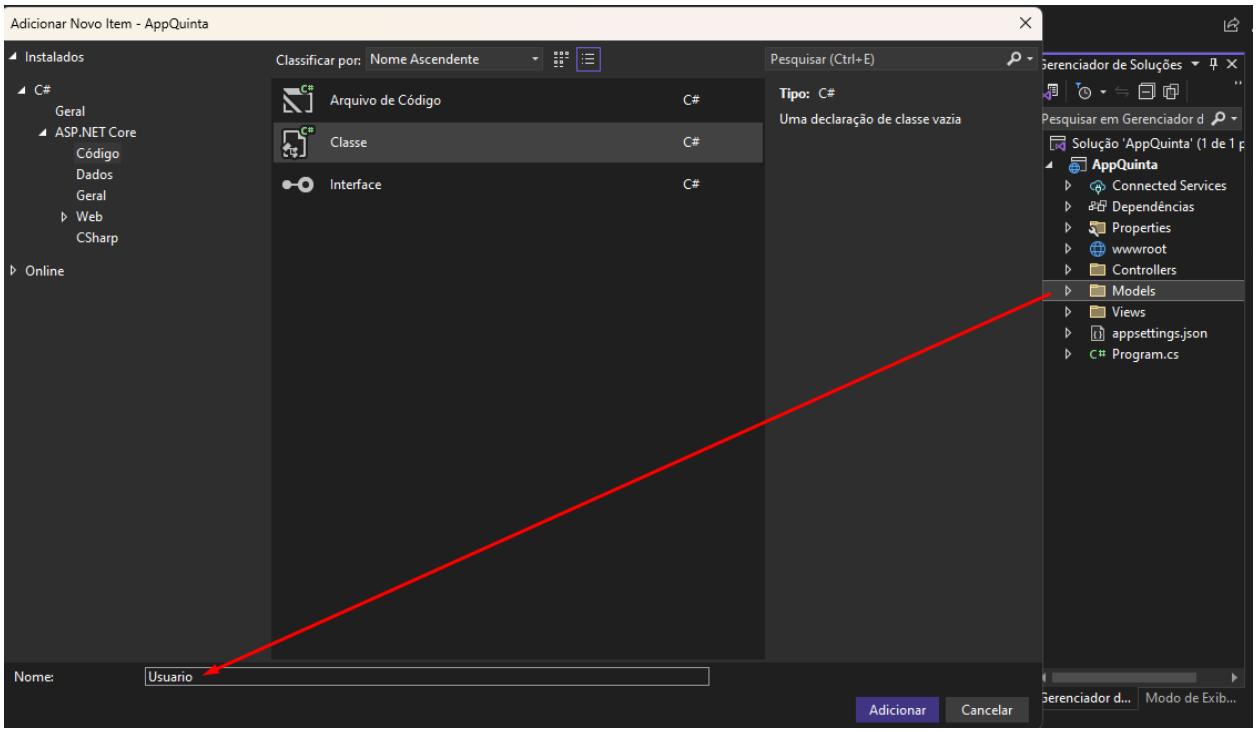


Vamos adicionar a String de conexão, no arquivo appsettings.json coloque sua conexão de acordo com seu banco de dados conforme imagem abaixo



Vamos adicionar uma classe Usuario dentro da pasta Model





Vamos colocar os atributos da classe

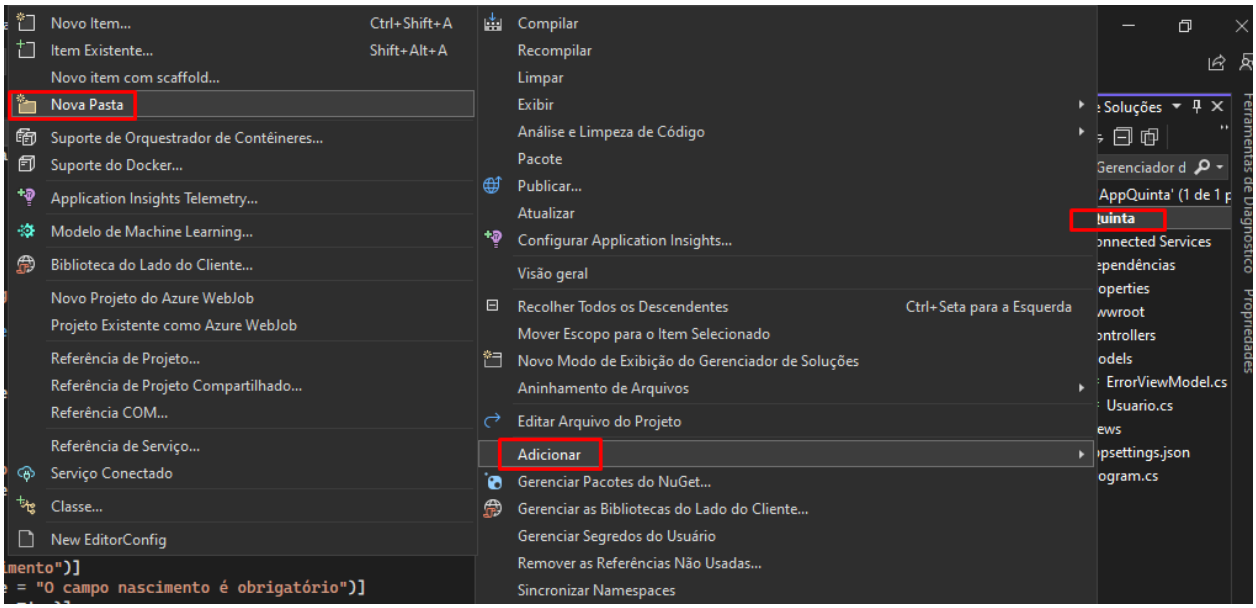
Vamos adicionar os atributos Data Annotations.

```
0 referências
public class Usuario
{
    [Display(Name = "Código")]
    0 referências
    public int? IdUsu { get; set; }

    [Display(Name = "Nome Completo")]
    [Required(ErrorMessage ="O campo nome é obrigatório")]
    0 referências
    public string nomeUsu { get; set; }

    [Display(Name = "Cargo")]
    [Required(ErrorMessage = "O campo Cargo é obrigatório")]
    0 referências
    public string Cargo { get; set; }

    [Display(Name = "Nascimento")]
    [Required(ErrorMessage = "O campo nascimento é obrigatório")]
    [DataType(DataType.DateTime)]
    0 referências
    public DateTime DataNasc { get; set; }
}
```

Vamos criar uma nova pasta Repository

Clicando com o botão direito sobre o nome do projeto depois adicionar nova pasta

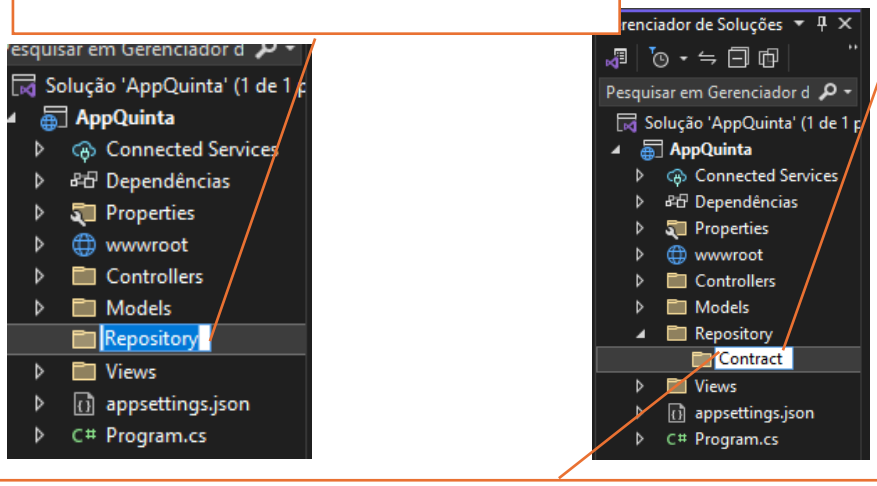
Dentro da pasta Repository criaremos outra pasta com o nome Contract



Os padrões de repositório e unidade de trabalho têm a intenção de criar uma camada de abstração entre a camada de acesso a dados e a camada de lógica de negócios de um aplicativo. Implementar esses padrões pode ajudar a isolar seu aplicativo de alterações no armazenamento de dados e pode facilitar testes de unidade automatizados ou desenvolvimento orientado a testes TDD(test-driven development). [1]

No **Repository** ficará as classes de manipulação de dados

No **Contract** ficará as Inteface com as assinaturas dos métodos

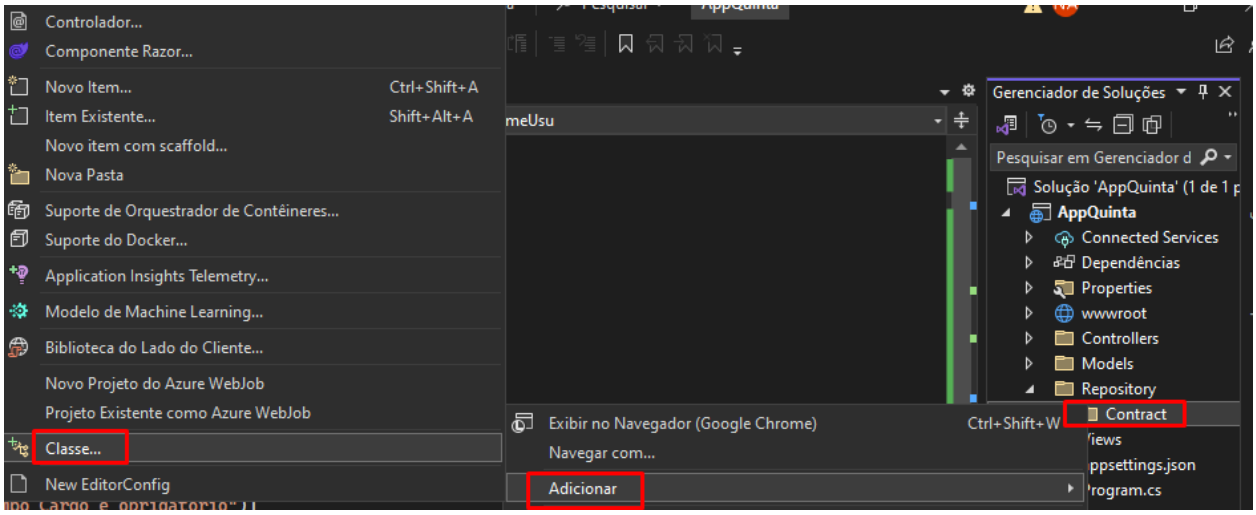


Uma abstração é um tipo que descreve um contrato, mas não fornece uma implementação completa do contrato. As abstrações geralmente são implementadas como classes abstratas ou interfaces e vêm com um conjunto bem definido de documentação de referência que descreve a semântica necessária dos tipos que implementam o contrato

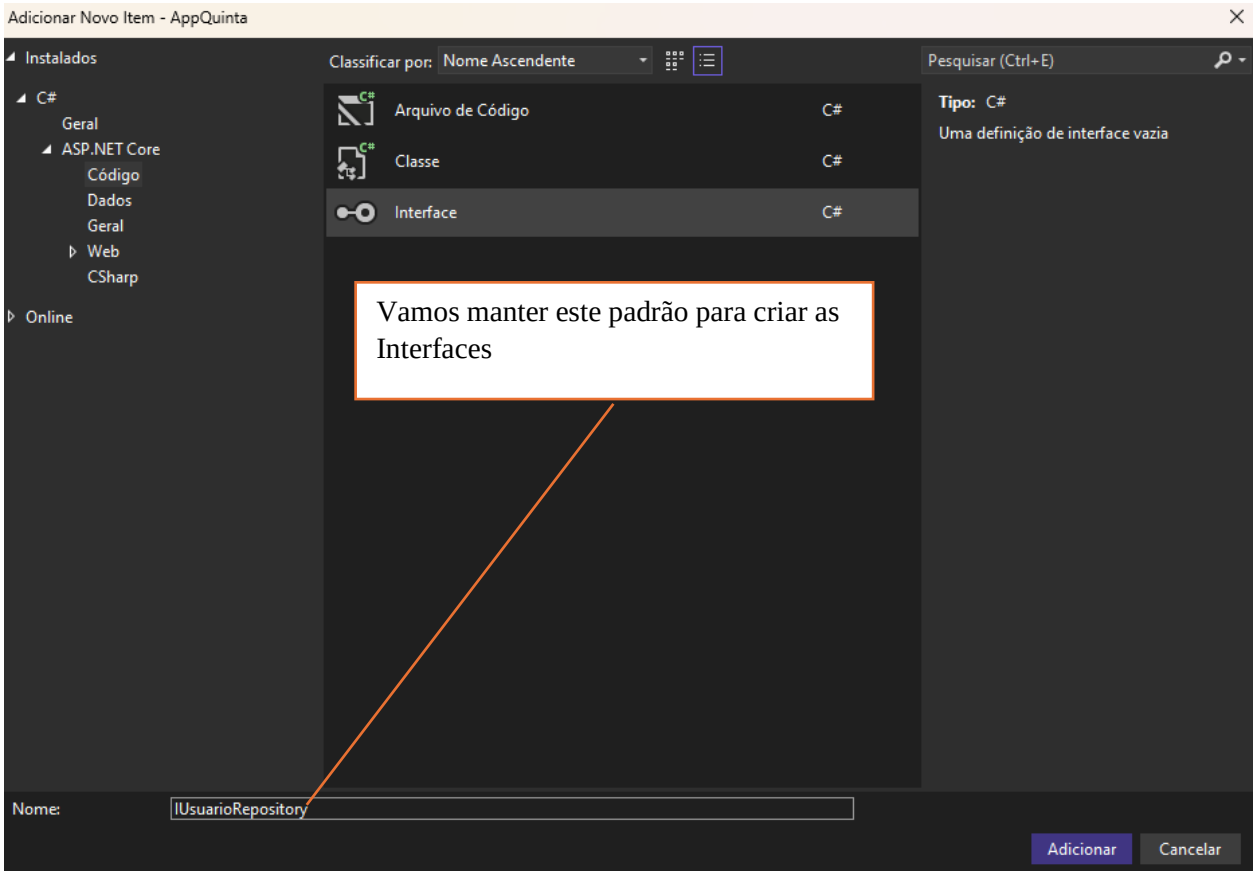
[1] https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions/getting-started-with-ef-5-using-mvc-4/implementing-the-repository-and-unit-of-work-patterns-in-an-asp-net-mvc-application
https://learn.microsoft.com/pt-br/dotnet/standard/design-guidelines/abstractions-abstract-types-and-interfaces

Vamos criar a IUsuarioRepository interface onde colocaremos o CRUD

Clicando com o botão direito sobre a pasta Contract conforme imagem abaixo



Vamos escolher Interface



IEnumerable retorna uma lista de usuários

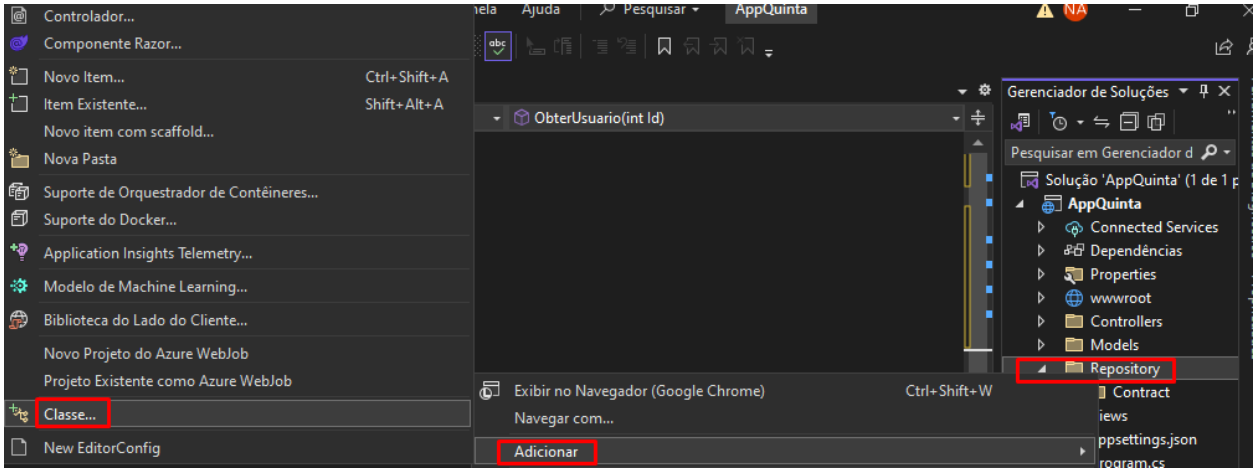
Void não retorna nada

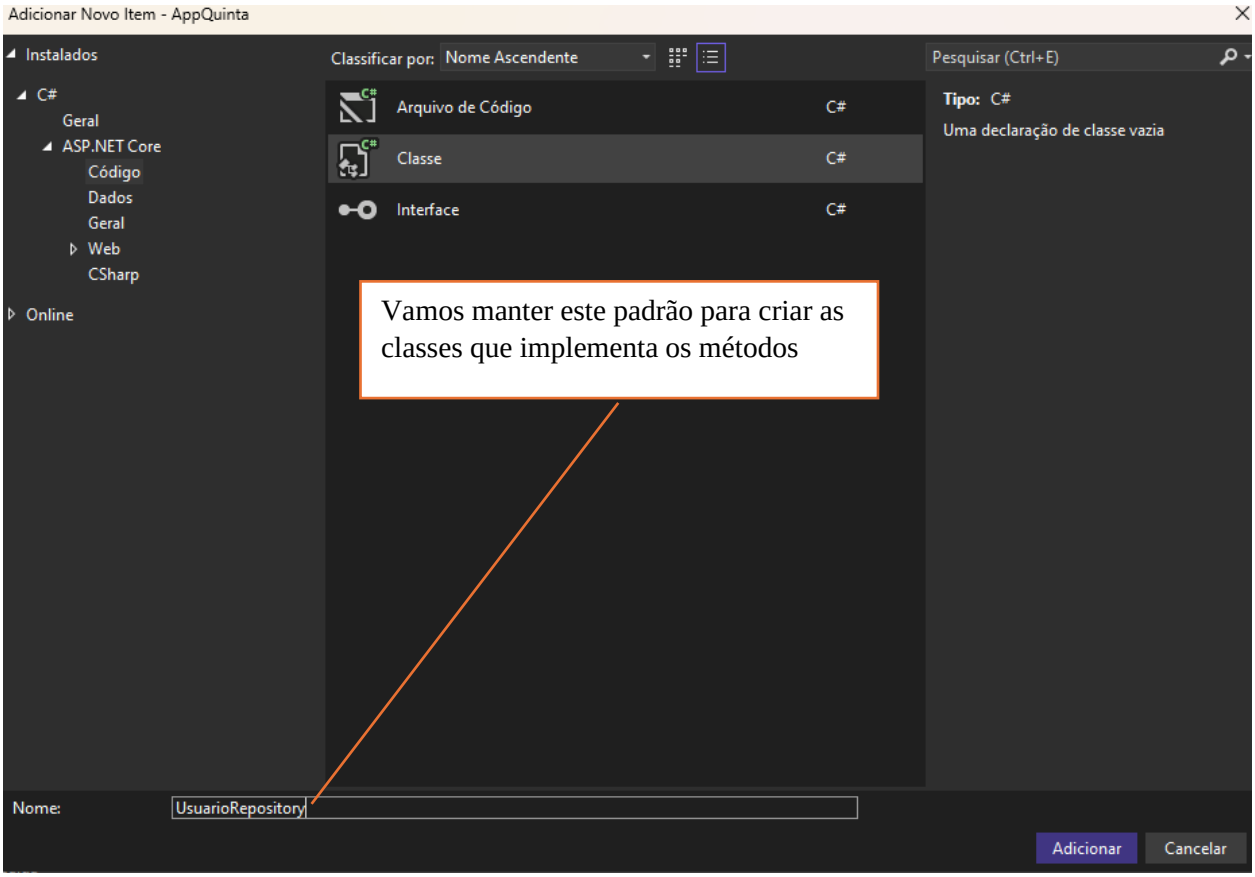
Retorna o objeto (Usuario) passando como parâmetro o Id

```
0 referências
public interface IUsuarioRepository
{
    //CRUD
    0 referências
    IEnumerable<Usuario> ObterTodosUsuarios();

    0 referências
    void Cadastrar(Usuario usuario);

    0 referências
    void Atualizar(Usuario usuario);

    0 referências
    Usuario ObterUsuario(int Id);

    0 referências
    void Excluir(int Id);
}
```

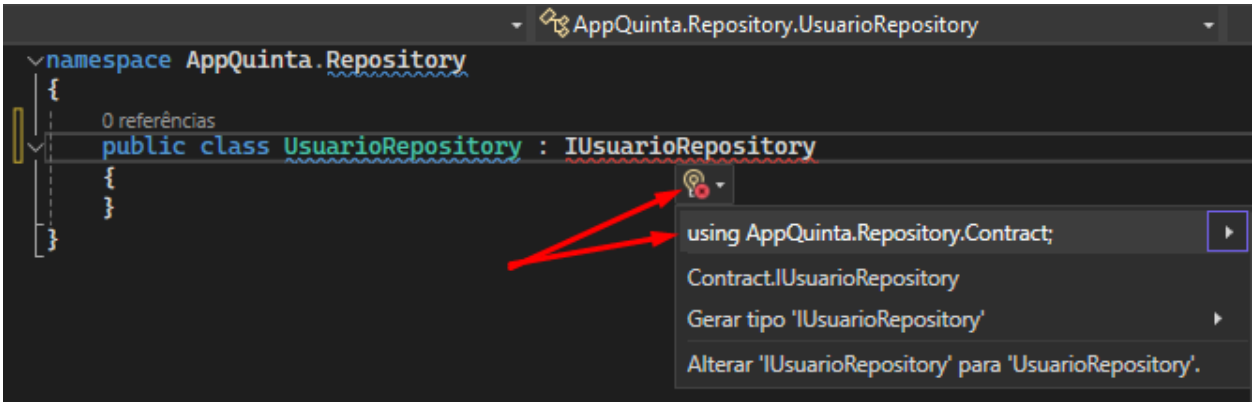
Criaremos agora a classe UsuarioRepository esta classe é responsável por implementar os contratos da interface, ou seja, reescrever os métodos de acordo com a nossa necessidade.

Clicando com o botão direito sobre a pasta repository conforme imagem abaixo.

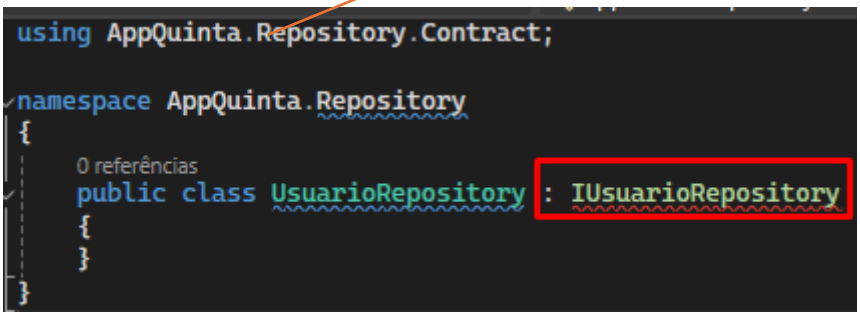




Após criar a classe implementar a interface para termos acesso aos metodos e então reescreve-los

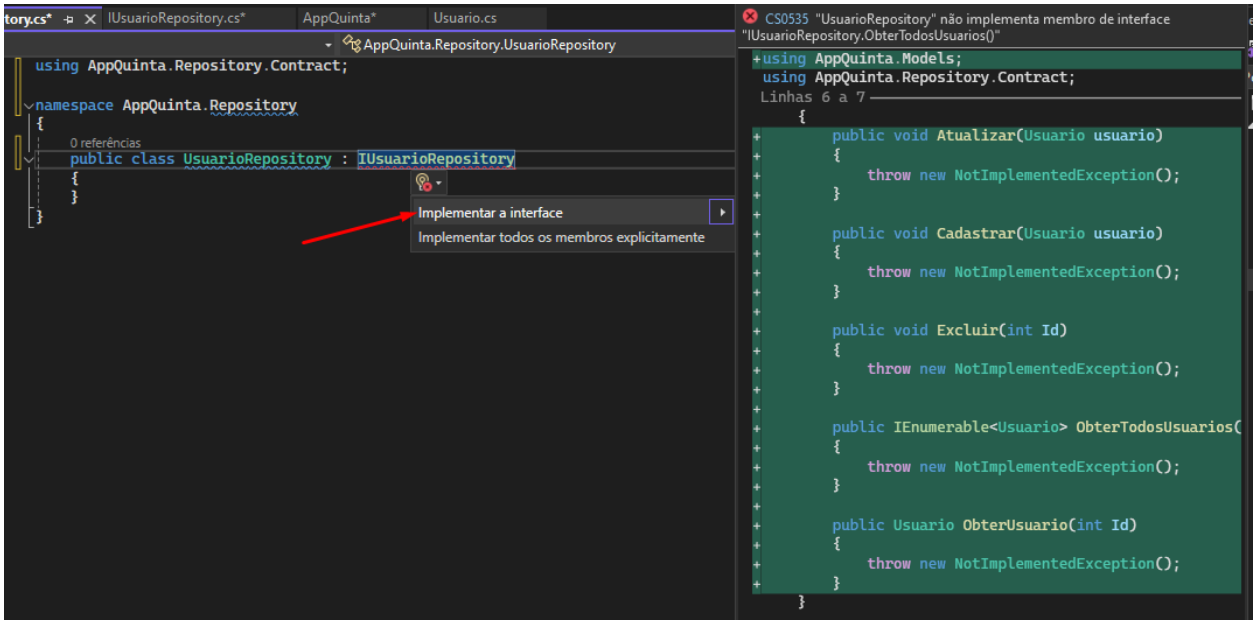


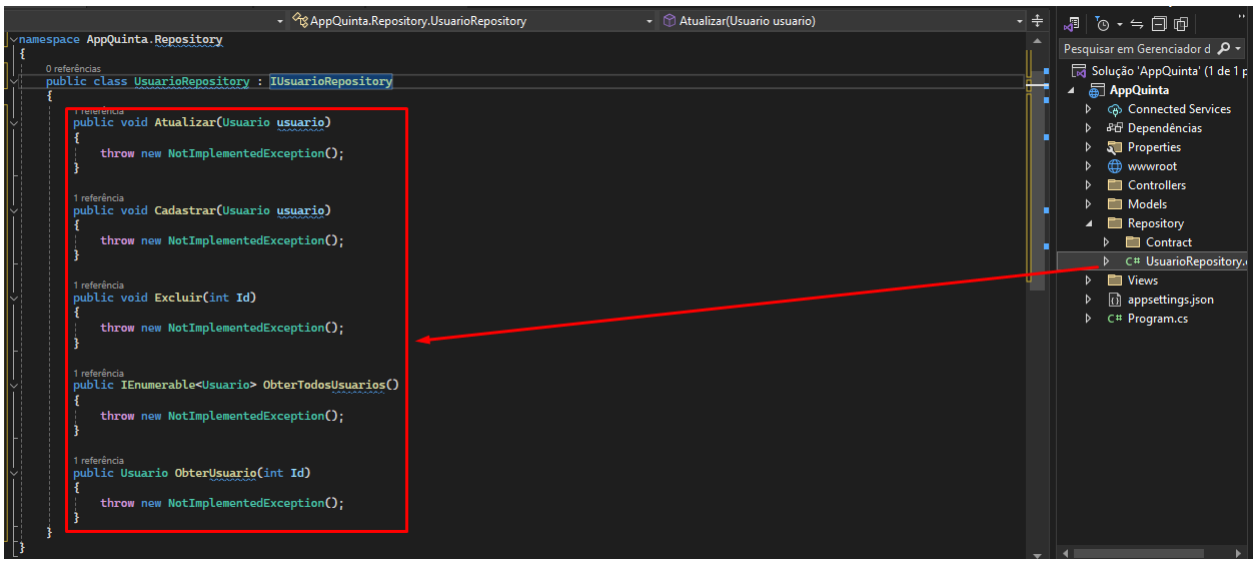
Para ter acesso aos contratos na interface precisamos usar o namespace



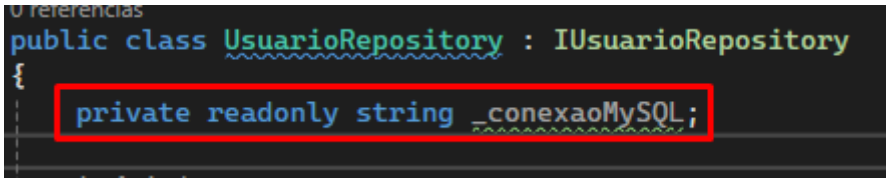
Agora podemos acessar os contratos.

Passando o mause sobre IUsuarioRepository clica no balãozinho depois em implementar interface





Vamos criar uma propriedade privada para conectar com o banco de dados



Um construtor é um método cujo nome é igual ao nome de seu tipo. Sua assinatura do método inclui apenas um modificador de acesso opcional, o nome do método e sua lista de parâmetros; ela não inclui tipo de retorno

Criamos o método construtor da classe passando a conexão como parâmetro.[2]

```
1 referência
public class UsuarioRepository : IUsuarioRepository
{
    private readonly string _conexaoMySQL;

    0 referências
    public UsuarioRepository(IConfiguration conf)
    {
        _conexaoMySQL = conf.GetConnectionString("ConexaoMySQL");
    }
```

Paramos aqui

Quando passamos o parâmetro IConfiguration temos acesso ao "appsettings" e podemos pegar a string de conexão com o método GetConnectionString

ConnectionString

Na computação, uma string de conexão é uma string que especifica informações sobre uma fonte de dados e os meios de se conectar a ela. Ele é passado em código para um driver ou provedor subjacente para iniciar a conexão. Embora comumente usados para uma conexão de banco de dados, a fonte de dados também pode ser uma planilha ou arquivo de texto. [3]

```
"AllowedHosts": "*",
"ConnectionStrings": {
  "ConexaoMySQL": "server=localhost;port=3307;user id=root;password=361190;database=dbBancoApp"
}
```

Links interessantes sobre strings de conexão:

http://www.linhadecodigo.com.br/artigo/1640/connection-strings-para-diversos-bancos-de-dados.aspx

https://www.connectionstrings.com/

Veja exemplos de conexões:

Sql Server

Data Source=meuServidor; Initial Catalog=meuBanco; User Id=NomeUsuario; Password=SenhaUsuario;

MySql

Server=meuServidor; Database=meuBanco; Uid=NomeUsuario; Pwd=SenhaUsuario ;

Em alguns casos, será preciso informar o parâmetro da porta, por exemplo "Port=3306". No caso do MySql a porta padrão é a 3306 logo não precisa ser informado, mas se o SGBD estiver utilizando outra porta, essa deve ser informada.

---

[2] https://learn.microsoft.com/pt-br/dotnet/csharp/programming-guide/classes-and-structs/constructors
https://www.macoratti.net/18/09/c_constr1.htm

[3] https://en.wikipedia.org/wiki/Connection_string

O que é um objeto Connection

- Um objeto Connection representa uma conexão física com uma fonte de dados.
- Para criar um objeto Connection devemos fornecer o nome de um provedor OLE DB ou o nome de uma fonte de dados ODBC.
- Quando abrimos um objeto Connection estamos tentando a conexão com a fonte de dados.
- A propriedade State do objeto Connection nos dirá se a conexão foi um sucesso ou se falhou.
- Podemos enviar instruções SQL ou rodar stored procedures usando o método Execute do objeto Connection.
- Se a instrução que enviamos ao objeto Connection retornar um conjunto de registros um objeto Recordset será criado automaticamente. [4]

Veja os métodos e propriedades do objeto connection:

Método	Descrição
Open	Abre uma conexão com uma fonte de dados
Close	Fecha a conexão e todos os objetos dependentes dela.
Execute	Executa uma consulta , instrução SQL , stored procedures ou um comando específico do provedor
BeginTrans	Inicia uma nova Transação
CommiTrans	Salva qualquer alteração e encerra a transação atual.
RollBackTrans	Cancela qualquer alteração feita durante a transação atual e encerra a transação.

Propriedade	Descrição
ConnectionString	Contém a informação usada para estabelecer uma conexão com uma fonte de dados
ConnectionTimeout	Indica o tempo de espera durante a tentativa de uma conexão , antes de terminar e gerar um erro
CommandTimeout	Indica o tempo de espera durante a execução de um comando antes de terminar e gerar um erro
State	Indica se a conexão esta aberta, fechada ou em andamento
Provider	Informa o nome do provedor usado na conexão

Data Source

Uma fonte de dados é um local onde as informações são coletadas. A origem pode ser um banco de dados, um arquivo simples, um documento XML ou qualquer outro formato que um sistema possa ler. A entrada é capturada como um conjunto de registros contendo informações utilizadas no fluxo de trabalho. Informações do cliente, números contábeis, vendas, logística e outras informações podem ser incluídas. [5]

Para criar a nossa conexão, vamos precisar de um objeto connection. No exemplo a seguir com o MySql "MySqlConnection", se fosse com o Sql Server seria "SqlConnection".

[4] https://docs.microsoft.com/pt-br/sql/ado/reference/ado-api/state-property-ado?view=sql-server-2017
http://www.macoratti.net/connect.htm
[5] https://www.questionpro.com/blog/data-source/

Abrindo a conexão com o banco

Já sabemos que o conteúdo da propriedade "ConnectionSttring" pode ser passado como parâmetro!

```
public void Cadastrar(Usuario usuario)
{
    using( var conexao = new MySqlConnection(_conexaoMySQL))
    {
        conexao.Open();

        MySqlCommand cmd = new MySqlCommand("insert into usuario(nomeUsu, Cargo, DataNasc) " +
                                            " values (@nomeUsu, @Cargo, @DataNasc )", conexao); //

        cmd.Parameters.Add("@nomeUsu", MySqlDbType.VarChar).Value = usuario.nomeUsu;
        cmd.Parameters.Add("@Cargo", MySqlDbType.VarChar).Value = usuario.Cargo;
        cmd.Parameters.Add("@DataNasc", MySqlDbType.VarChar).Value = usuario.DataNasc;

        cmd.ExecuteNonQuery();
        conexao.Close();
    }
}
```

Criando um objeto comando

```
ToString("yyyy/mm/dd");
```

Acrescentar

ExecuteNonQuery() – Execute um comando do tipo Update, Insert, Delete

Parameters – É uma coleção de parâmetros associada a um comando

Fecha a conexão com o banco

Objeto Command

O padrão Command é bastante comum no código C#. Na maioria das vezes, é usado como uma alternativa para retornos de chamada para parametrizar elementos da interface do usuário com ações. Também é usado para enfileirar tarefas, rastrear o histórico de operações etc.

Propriedade do objeto Command:

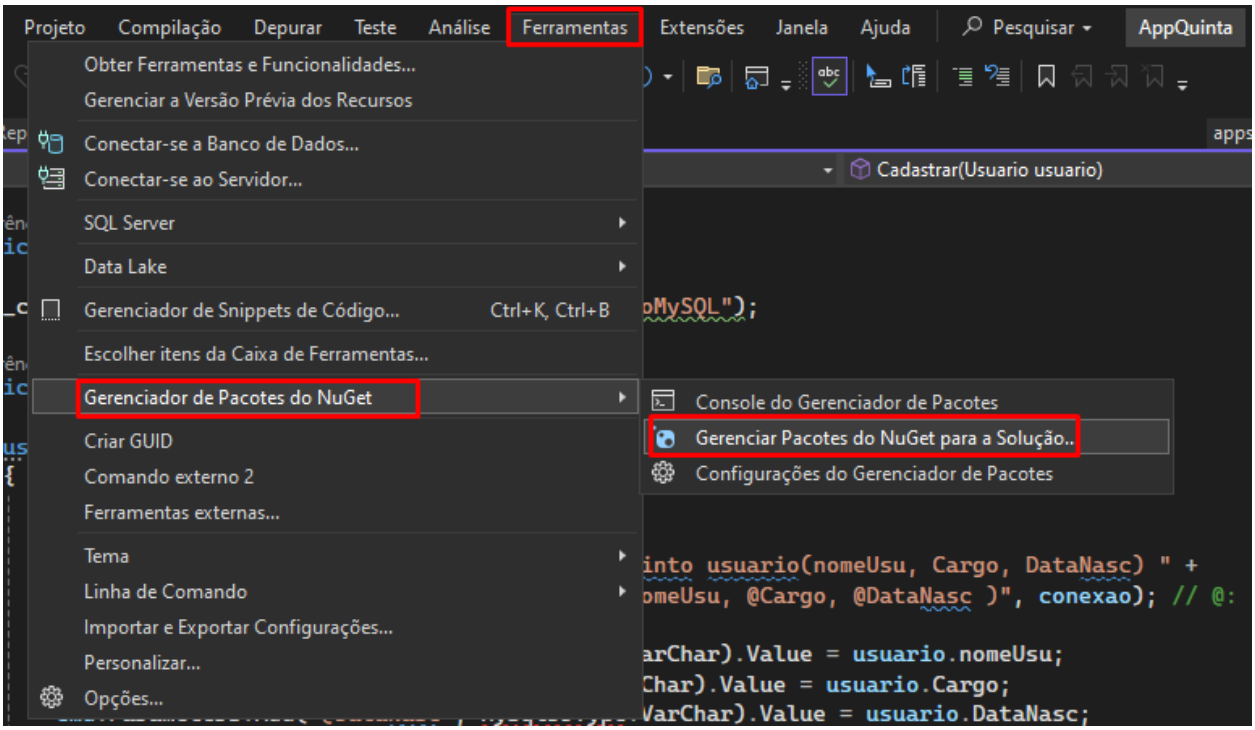
- **CommandText** – Comando SQL ou nome da Stored Procedure a chamar
- **CommandType** – Determina o tipo de CommandText
- **Parameters** – É uma coleção de parâmetros associada a um comando

Métodos:

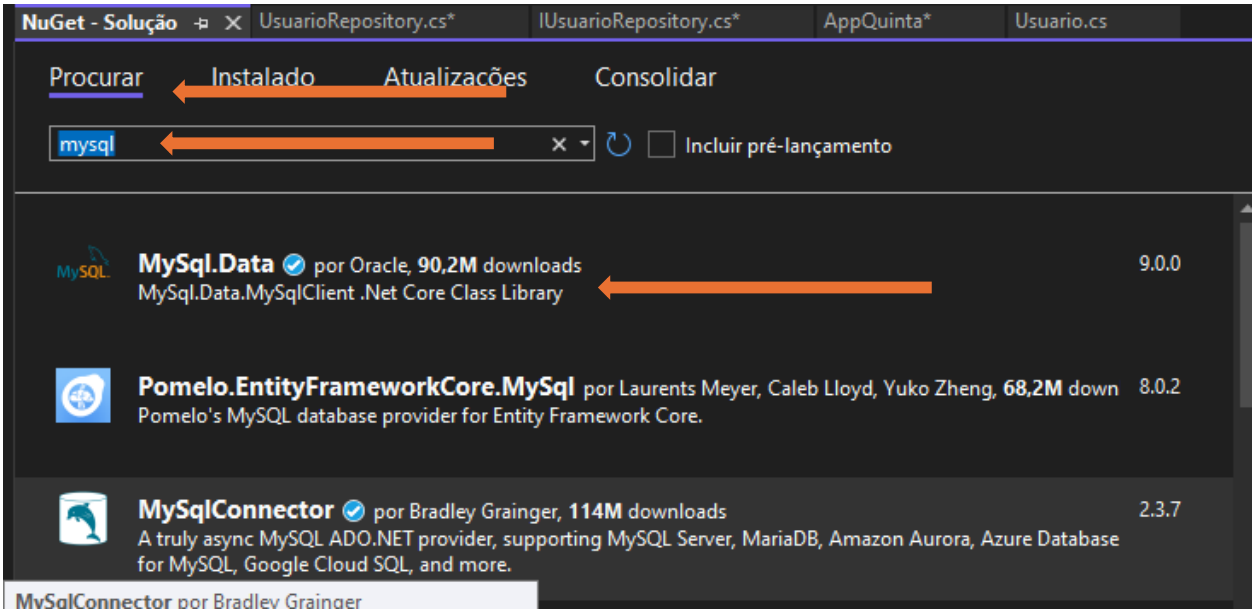
- **ExecuteReader()** – executa um SELECT e retorna um bloco de Registros
- **ExecuteScalar()** – executa um SELECT e retorna um valor apenas
- **ExecuteNonQuery()** – Execute um comando do tipo Update, Insert, Delete

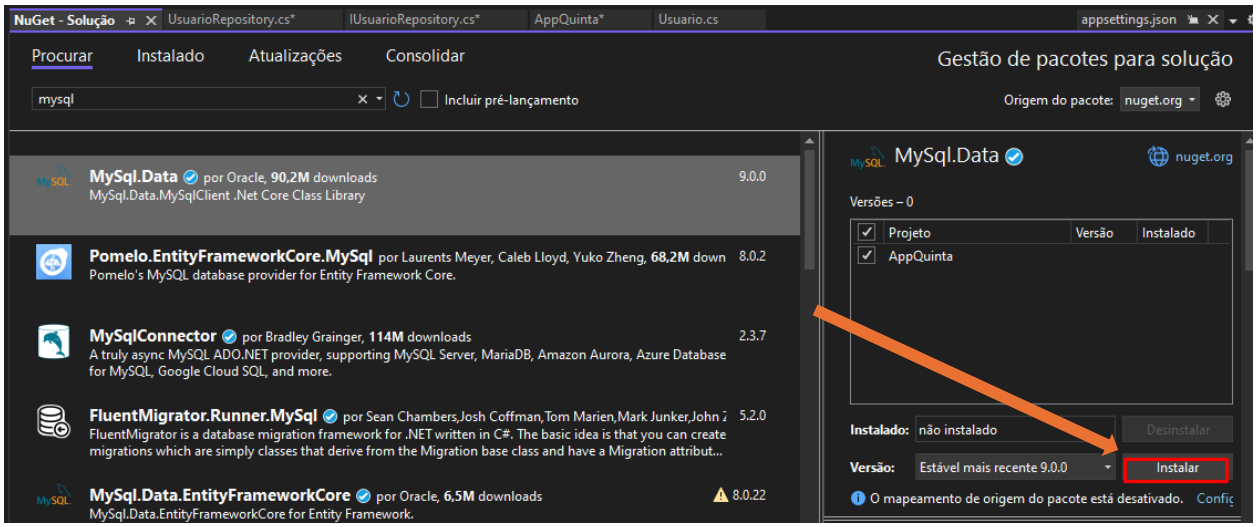
Para corrigir os erros acima vamos instalar o pacote MySQL

Clica em ferramenta – Gerenciamento de pacote do nuget – gerenciaaar pacote de Nuget ....

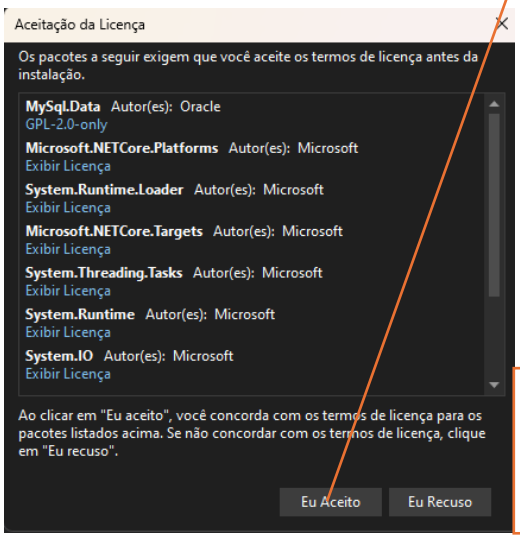


Na aba “NuGet:”, selecione “Procurar” e digite “MySql.Data”, o browser irá procurar o pacote, selecione e clique em “Instalar”.

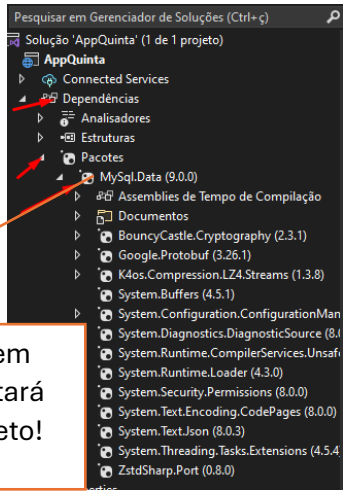




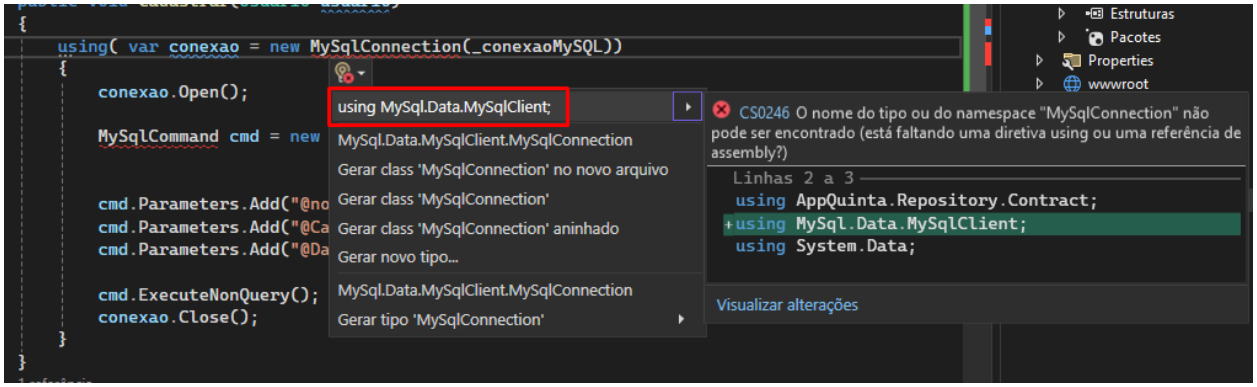
Para realizar a instalação, você deve ler os termos da licença e clicar em aceitar.

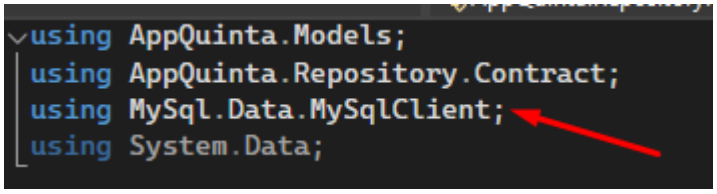


Após a instalação, observe que em referências, o “MySql.Data” já estará disponível para utilização no projeto!



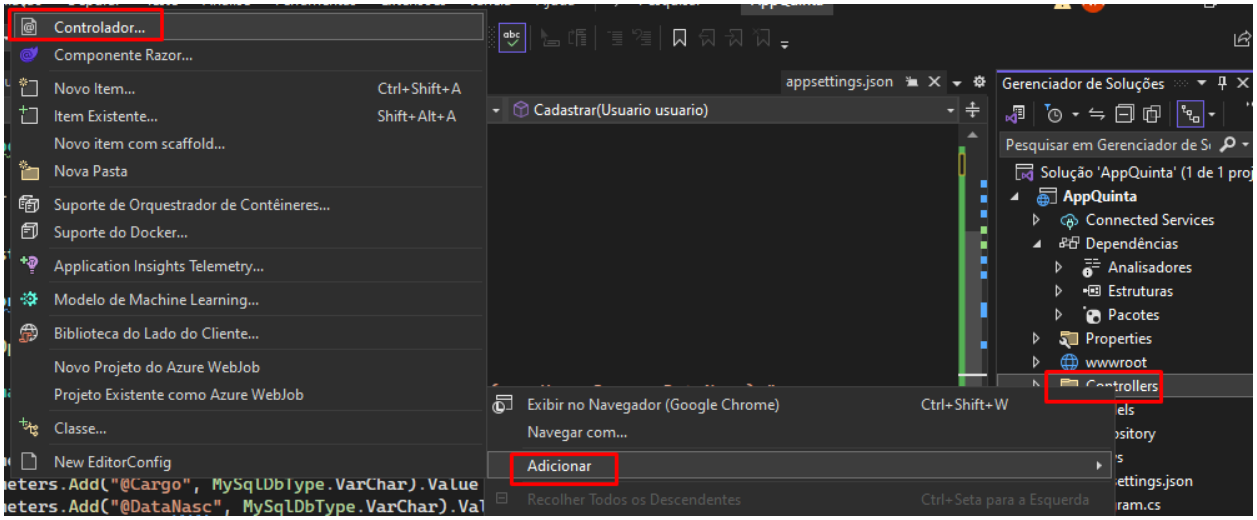
Para reconhecer os comando MySQL vamos adicionar a diretiva MySqlClient



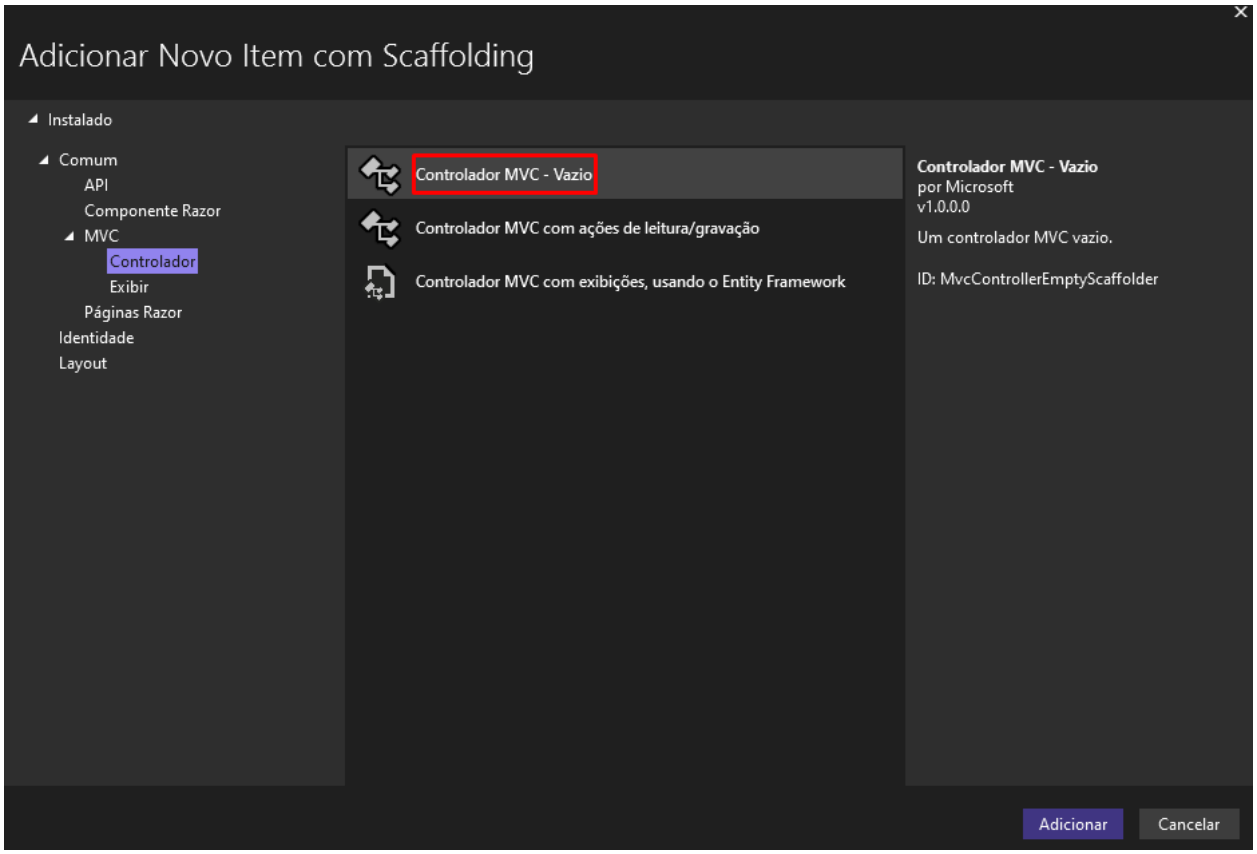


Vamos adicionar um controlador UsuarioController

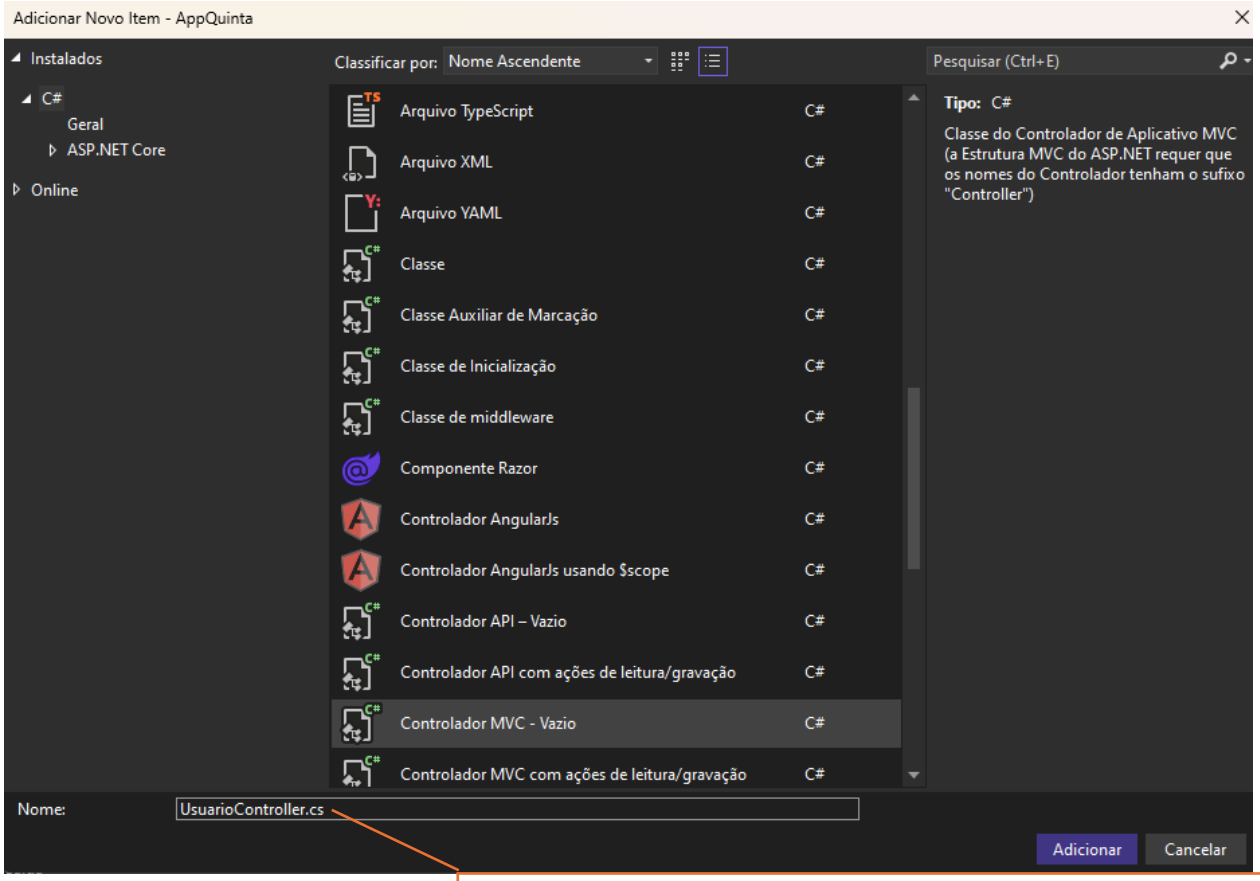
Clicando com o botão direito sobre Controller conforme imagem abaixo



Vamos adicionar um controller vazio

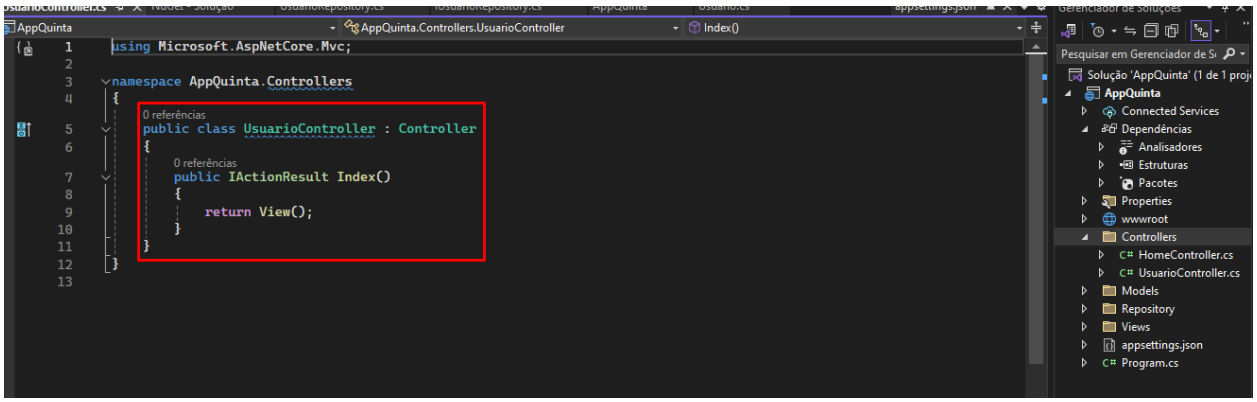


Vamos dar o nome de UsuarioController



Nome do Controller < Nome+sufixo: Toda classe Controller, tem que terminar com o sufixo Controller.

Lembre-se! Controllers = Classes responsáveis de trata as requisições web



Injeção de dependência

A injeção de dependência, DI (Dependency Injection) é um dos pilares do desenvolvimento de aplicações em ASP.NET Core. Ela permite a criação de um sistema mais modular e testável, desacoplando a implementação de classes das suas dependências.

No ASP.NET Core temos este recurso fortemente adicionado de forma generica

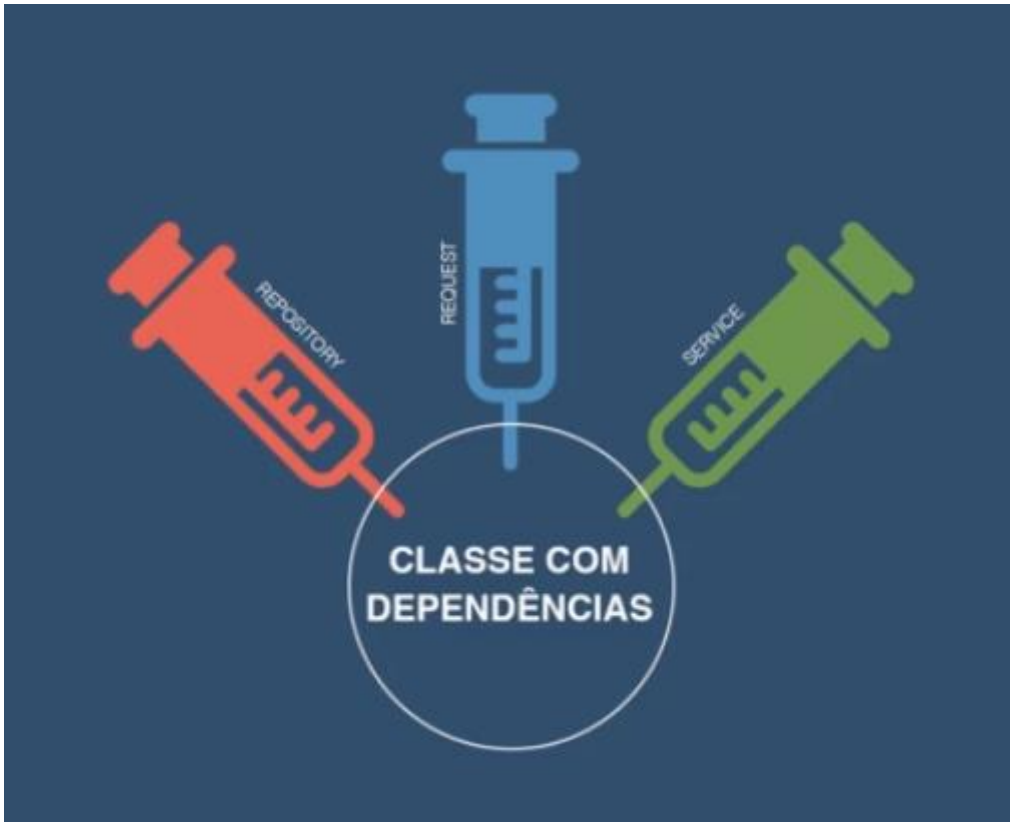
Para usar a injeção de dependência precisamos adicionar na Program.cs como serviços.

Existem três modos principais de registrar serviços:

Transient: Uma nova instância é criada cada vez que o serviço é solicitado.

Scoped: Uma nova instância é criada por escopo de solicitação.

Singleton: Uma única instância é criada e usada durante toda a vida útil da aplicação



Vamos adicionar a IUsuarioRepositorio com serviço na classe Program.cs

```
using AppQuinta.Repository;
using AppQuinta.Repository.Contract;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

// Adicionando a interface como serviço
builder.Services.AddScoped<IUsuarioRepository, UsuarioRepository>();

var app = builder.Build();
```

Agora podemos injetar a dependência de acordo com a classe, vamos injetar no UsuarioController

Para isso temos que definir uma propriedade privada.

```
0 referências
public class UsuarioController : Controller
{
    private IUsuarioRepository _usuarioRepository;
    0 referências
    public IActionResult Index()
    {
        return View();
    }
}
```

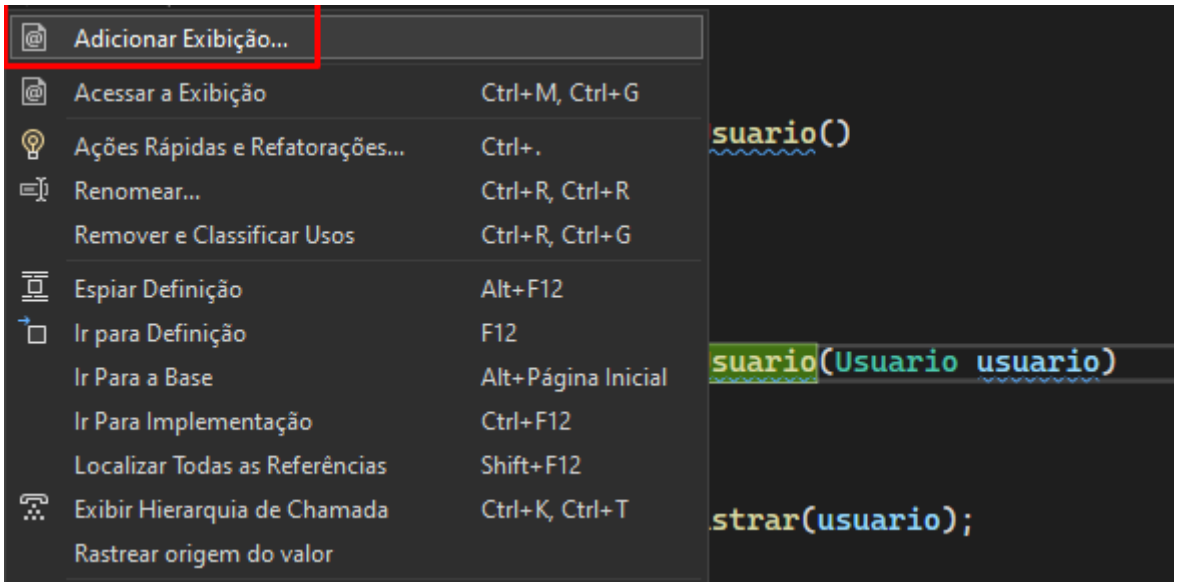
Vamos criar um método construtor do UsuarioController

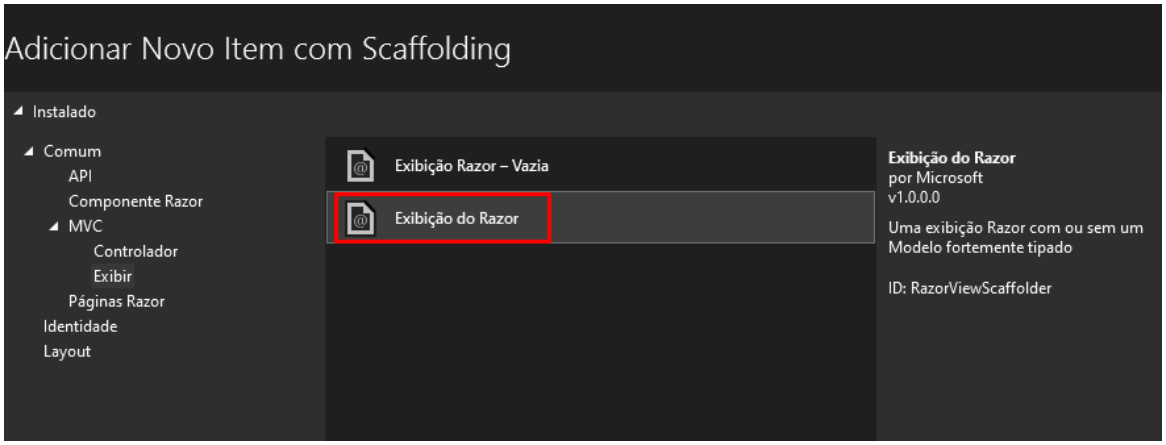
```
namespace AppQuinta.Controllers
{
    1 referência
    public class UsuarioController : Controller
    {
        private IUsuarioRepository _usuarioRepository;

        0 referências
        public UsuarioController (IUsuarioRepository usuarioRepository)
        {
            _usuarioRepository = usuarioRepository;
        }
        0 referências
        public IActionResult Index()
        {
            return View();
        }
    }
}
```

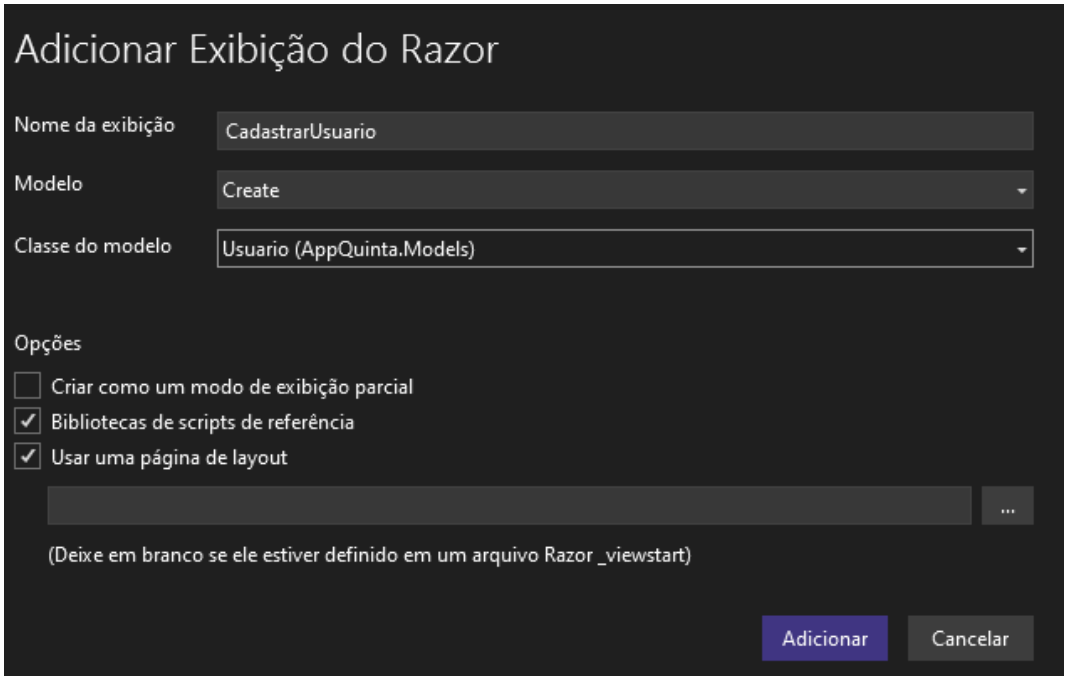
Criamos as actions para cadastrar usuário

```
[HttpGet]
0 referências
public IActionResult CadastrarUsuario()
{
    return View();
}
[HttpPost]
0 referências
public IActionResult CadastrarUsuario(Usuario usuario)
{
    if(ModelState.IsValid)
    {
        _usuarioRepository.Cadastrar(usuario);

    }
    return View();
}
```

Agora vamos criar a View para cadastrar





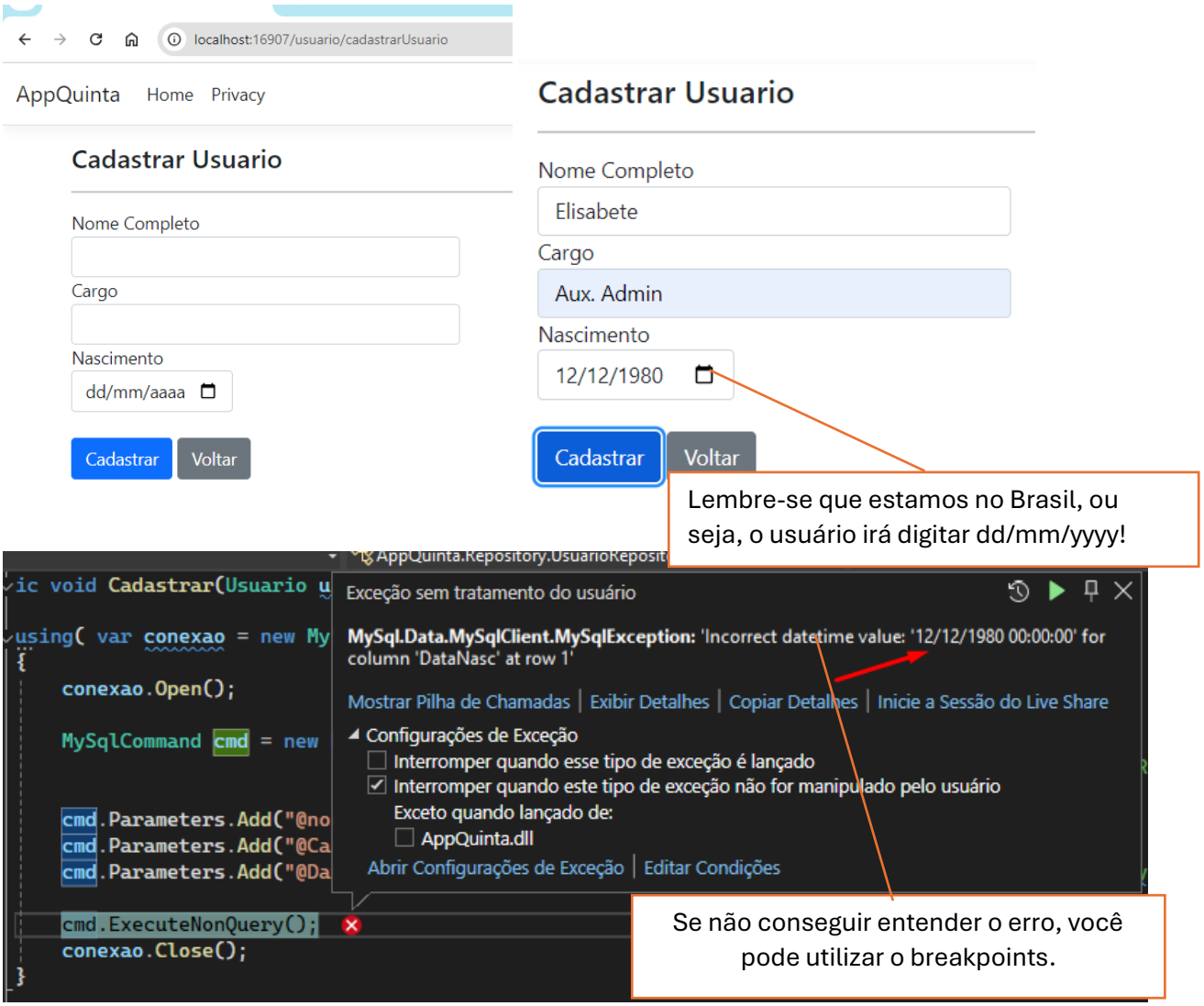


Vamos apagar a primeira div, pois o Id é auto incremente e não precisamos passar

```
<div class="row">
    <div class="col-md-4">
        <form asp-action="CadastrarUsuario">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="IdUsu" class="control-label"></label>
                <input asp-for="IdUsu" class="form-control" />
                <span asp-validation-for="IdUsu" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="nomeUsu" class="control-label"></label>
                <input asp-for="nomeUsu" class="form-control" />
                <span asp-validation-for="nomeUsu" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Cargo" class="control-label"></label>
                <input asp-for="Cargo" class="form-control" />
                <span asp-validation-for="Cargo" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="DataNasc" class="control-label"></label>
                <input asp-for="DataNasc" class="form-control" />
                <span asp-validation-for="DataNasc" class="text-danger"></span>
            </div>
            <div class="form-group">
                <input type="submit" value="Cadastrar" class="btn btn-primary" />
            </div>
        </form>
```

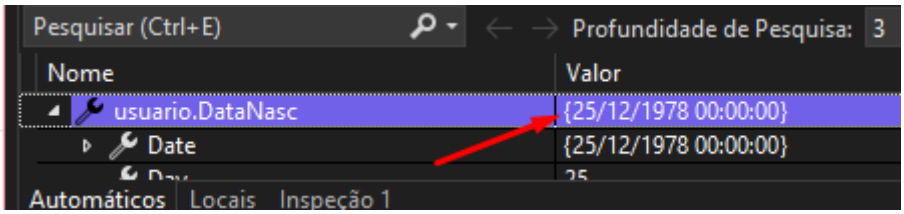
Não foi encontrado nenhum problema

```
@model AppQuinta.Models.Usuario

@{    ViewData["Title"] = "CadastrarUsuario"; }

<h4>Cadastrar Usuario</h4>  <hr />
<div class="row">
    <div class="col-md-4">
        <form asp-action="CadastrarUsuario">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="nomeUsu" class="control-label"></label>
                <input asp-for="nomeUsu" class="form-control" />
                <span asp-validation-for="nomeUsu" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Cargo" class="control-label"></label>
                <input asp-for="Cargo" class="form-control" />
                <span asp-validation-for="Cargo" class="text-danger"></span>
            </div>
            <div  class="col-5 form-group">
                <label asp-for="DataNasc" class="control-label"></label>
                <input type="date" asp-for="DataNasc" class="form-control" />
                <span asp-validation-for="DataNasc" class="text-danger"></span>
            </div>
            <br />
            <div class="form-group">

                <input type="submit" value="Cadastrar" class="btn btn-primary" />

                <a class="btn btn-secondary" asp-action="Index">Voltar</a>
            </div>
        </form>
    </div>
</div>

@section Scripts {  @{await Html.RenderPartialAsync("_ValidationScriptsPartial");} }
```



Analisando o problema com Breakpoints

Os pontos de interrupção são uma das técnicas de depuração mais importantes na caixa de ferramentas de seu desenvolvedor. Você define pontos de interrupção sempre que quiser pausar a execução do depurador. Por exemplo, você pode querer ver o estado das variáveis de código ou observar a pilha de chamadas em um determinado ponto de interrupção.



Formato de data e hora - ISO 8601

A ISO 8601 aborda essa incerteza estabelecendo uma maneira internacionalmente aceita de representar datas: **"AAAA-MM-DD".** [6]

[6] https://www.iso.org/iso-8601-date-and-time-format.html

O padrão ISO 8601 define um formato reconhecido internacionalmente para representar datas e horas. Para tempos, eles são expressos com a notação horas-minutos-segundos. No caso de datas, o formato é ano-mês-dia. Este formato básico permite que os números sejam escritos diretamente após o outro. Para maior clareza, eles também podem ser separados por delimitadores. [7]

Todos os SGBDs seguem o padrão ISO 8601 para realizar o armazenamento.

STR_TO_DATE('20/10/2022 14:30:50','%d/%m/%Y %H:%i:%s')

Função MySQL STR_TO_DATE()

A função STR_TO_DATE() retorna uma data baseada em uma string e um formato. [8]

Sintaxe

STR_TO_DATE(string, format)

Paramete	Description
string	Obrigatório. A string a ser formatada para uma data
format	Obrigatório. O formato a ser usado. Pode ser um ou uma combinação dos seguintes

Format	Description
%a	Abbreviated weekday name (Sun to Sat)
%b	Abbreviated month name (Jan to Dec)
%c	Numeric month name (0 to 12)
%D	Day of the month as a numeric value, followed by suffix (1st, 2nd, 3rd, ...)
%d	Day of the month as a numeric value (01 to 31)
%e	Day of the month as a numeric value (0 to 31)
%f	Microseconds (000000 to 999999)
%H	Hour (00 to 23)
%h	Hour (00 to 12)
%I	Hour (00 to 12)
%i	Minutes (00 to 59)
%j	Day of the year (001 to 366)
%k	Hour (0 to 23)
%l	Hour (1 to 12)
%M	Month name in full (January to December)
%m	Month name as a numeric value (01 to 12)
%p	AM or PM
%r	Time in 12 hour AM or PM format (hh:mm:ss AM/PM)
%S	Seconds (00 to 59)
%s	Seconds (00 to 59)
%T	Time in 24 hour format (hh:mm:ss)
%U	Week where Sunday is the first day of the week (00 to 53)
%u	Week where Monday is the first day of the week (00 to 53)
%V	Week where Sunday is the first day of the week (01 to 53). Used with %X
%v	Week where Monday is the first day of the week (01 to 53). Used with %X
%W	Weekday name in full (Sunday to Saturday)
%w	Day of the week where Sunday=0 and Saturday=6
%X	Year for the week where Sunday is the first day of the week. Used with %V
%x	Year for the week where Monday is the first day of the week. Used with %V
%Y	Year as a numeric, 4-digit value
%y	Year as a numeric, 2-digit value

Exemplo de formatação de data “Brasil”

STR_TO_DATE('10/10/2023','%d/%m/%Y')

Exemplo de formatação de data e hora “Brasil”

STR_TO_DATE('20/10/2022 14:30:50','%d/%m/%Y %H:%i:%s')

ou

STR_TO_DATE('20/10/2022 14:30:50','%d/%m/%Y %T')

Vamos criar um método no UsuarioRepository para selecionar todos usuários do banco

[7] https://www.ionos.com/digitalguide/websites/web-development/iso-8601/
[8] https://www.w3schools.com/sql/func_mysql_str_to_date.asp

```
1 referência
public IEnumerable<Usuario> ObterTodosUsuarios()
{
    List<Usuario> UsuarioList = new List<Usuario>();
    using(var conexao = new MySqlConnection(_conexaoMySQL))
    {
        conexao.Open();
        MySqlCommand cmd = new MySqlCommand("Select * from usuario");

        MySqlDataAdapter da = new MySqlDataAdapter(cmd);
        DataTable dt = new DataTable();
        da.Fill(dt);

        conexao.Clone();

        foreach(DataRow dr in dt.Rows)
        {
            UsuarioList.Add(
                new Usuario
                {
                    IdUsu = Convert.ToInt32(dr["IdUsu"])
                    nomeUsu = (string)dr["nomeUsu"],
                    Cargo = (string)dr["Cargo"],
                    DataNasc = Convert.ToDateTime(dr["DataNasc"])

                });
        }
        return UsuarioList;
    }

}
```

MySqlDataReader – criando um objeto do tipo DataReader para receber os dados vindos do banco.

DataTable – Um DataTable é como um container que podemos usar para armazenar informações de praticamente qualquer fonte de dados, sendo composto por uma coleção de linhas (rows) e colunas (columns)

DataReader

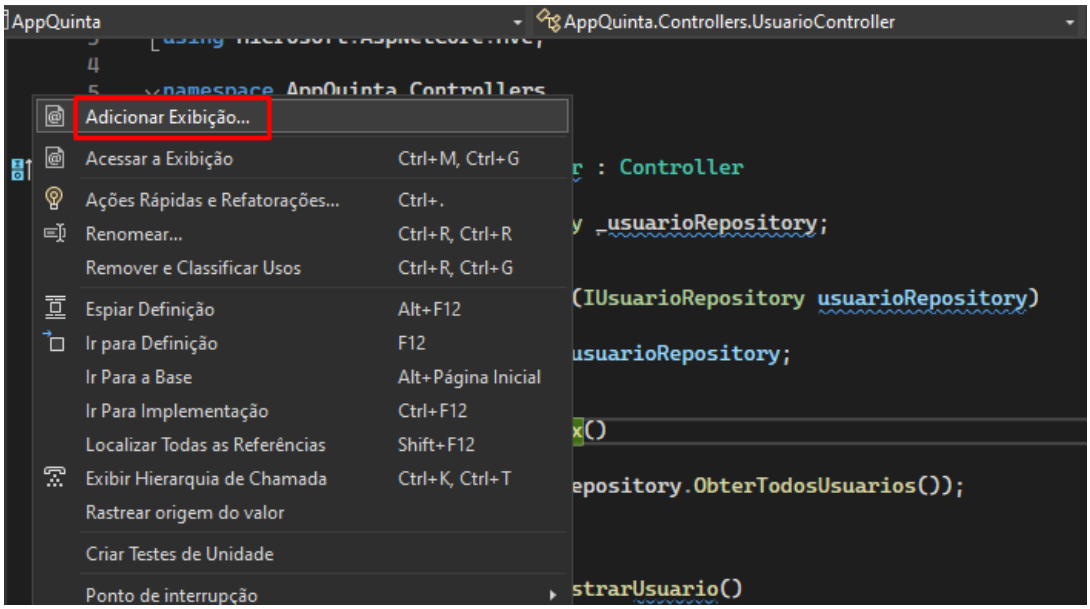
Um leitor de dados é um objeto usado para ler dados das fontes de dados. Isso só pode executar a operação de leitura e não a operação de atualização na fonte de dados. Os dados são recuperados como um fluxo de dados da fonte de dados. Embora o leitor de dados seja restrito em termos de operação apenas de leitura, ele é altamente eficaz e otimizado, pois é somente leitura e somente encaminhamento. Existem dois tipos de provedores no .Net Framework, eles são SQLDataReader e OleDbDataReader. O leitor de dados aumenta o desempenho do aplicativo reduzindo a sobrecarga do sistema, pois armazena a única linha na memória em um determinado momento. Este artigo abordará em detalhes o leitor de dados em c# junto com exemplos apropriados. [9]

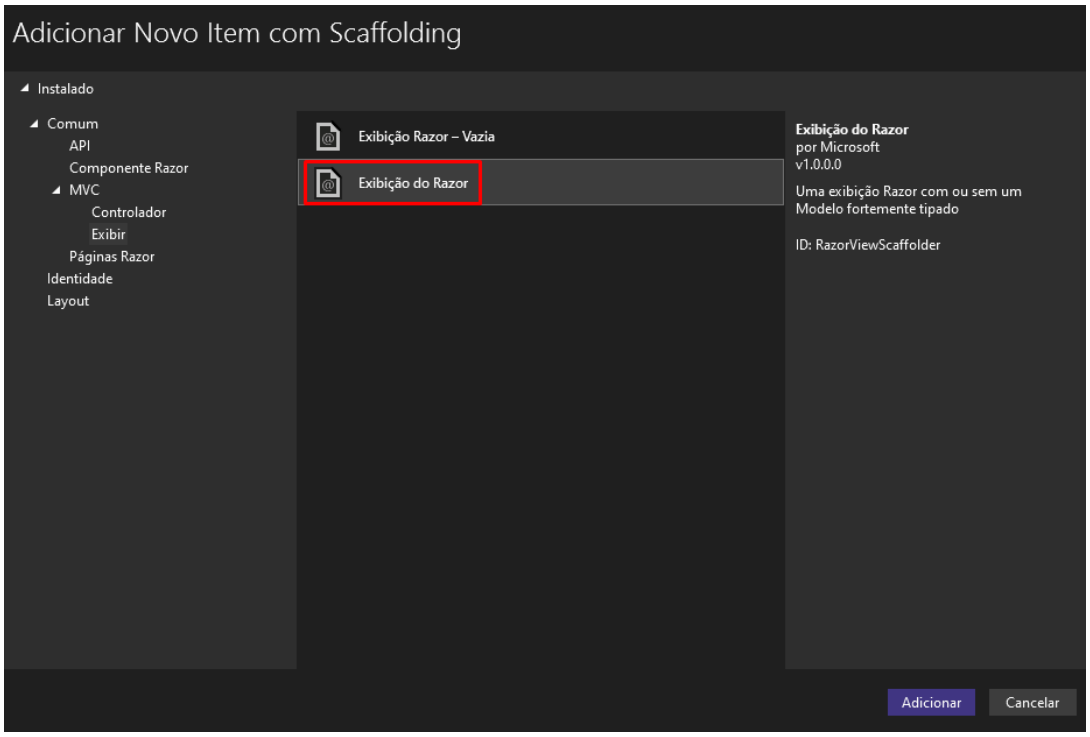
No UsuarioController vamos fazer a chamada deste método na Action Index

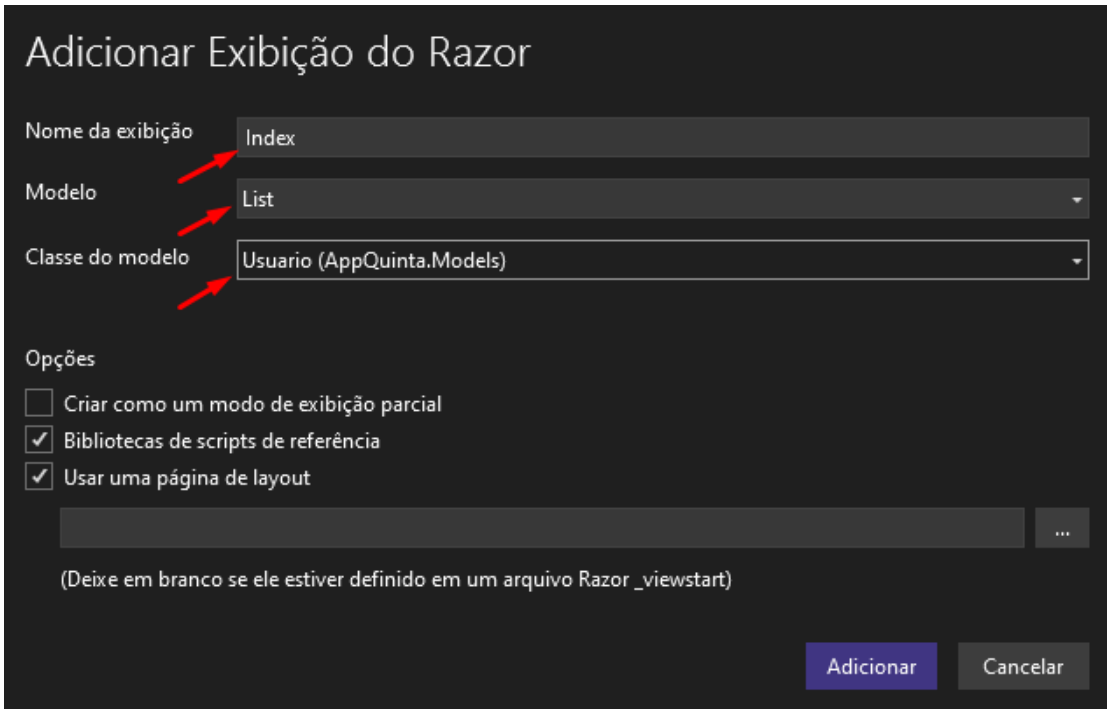
[9] https://www.educba.com/datareader-c-sharp/

```
1 referência
public class UsuarioController : Controller
{
    private IUsuarioRepository _usuarioRepository;

    0 referências
    public UsuarioController (IUsuarioRepository usuarioRepository)
    {
        _usuarioRepository = usuarioRepository;
    }
    0 referências
    public IActionResult Index()
    {
        return View(_usuarioRepository.ObterTodosUsuarios());
    }
    [HttpGet]
    0 referências
    public IActionResult CadastrarUsuario()
    {
        return View();
    }
```

Vamos adicionar a View Index.




Adicionar Novo Item com Scaffolding
Instalado
Comum
API
Componente Razor
MVC
Controlador
Exibir
Páginas Razor
Identidade
Layout
Exibição Razor – Vazia
Exibição do Razor
Exibição do Razor
por Microsoft
v1.0.0.0
Uma exibição Razor com ou sem um Modelo fortemente tipado
ID: RazorViewScaffolder
Adicionar
Cancelar


Adicionar Exibição do Razor
Nome da exibição
Index
Modelo
List
Classe do modelo
Usuario (AppQuinta.Models)
Opções
Criar como um modo de exibição parcial
Bibliotecas de scripts de referência
Usar uma página de layout
...
(Deixe em branco se ele estiver definido em um arquivo Razor _viewstart)
Adicionar
Cancelar

```
@model IEnumerable<AppQuinta.Models.Usuario>

@{
    ViewData["Title"] = "Index";
}
<p> <a class="btn btn-primary" asp-action="CadastrarUsuario" asp-controller="Usuario">Cadastrar</a></p>

<div class="table-responsive">
    <table class="table table-bordered">
        <thead>
            <tr>
                <th scope="col">@Html.DisplayNameFor(model => model.IdUsu)</th>
                <th scope="col">@Html.DisplayNameFor(model => model.nomeUsu) </th>
                <th scope="col">@Html.DisplayNameFor(model => model.Cargo) </th>
                <th scope="col">@Html.DisplayNameFor(model => model.DataNasc)</th>
                <th scope="col">Ação</th>
            </tr>
        </thead>
        <tbody>
            @foreach (Usuario usuario in Model)
            {
                <tr>
                    <td>@usuario.IdUsu</td>
                    <td>@usuario.nomeUsu </td>
                    <td>@usuario.Cargo</td>
                    <td>@usuario.DataNasc.ToString("dd/MM/yyyy")</td>
                    <td>
                        @Html.ActionLink("Edit", "Edit", new { /* id=item.PrimaryKey */ }) |
                        @Html.ActionLink("Details", "Details", new { /* id=item.PrimaryKey */ }) |
                        @Html.ActionLink("Delete", "Delete", new { /* id=item.PrimaryKey */ })
                    </td>
                </tr>
            }
        </tbody>
    </table>
</div>
```

AppQuinta Home Privacy

Cadastrar

Código	Nome Completo	Cargo	Nascimento	Ação
1	Nilson	Gerente	01/05/1978	Edit | Details | Delete
2	Bruno	Colaborador	12/10/2000	Edit | Details | Delete
3	Nilson José Alcides	Gerente	01/05/2005	Edit | Details | Delete

Vamos reescrever o método obterUsuario(int Id) dentro da classe UsuarioRepository

```
public Usuario ObterUsuario(int Id)
{
    using (var conexao = new MySqlConnection(_conexaoMySQL))
    {
        conexao.Open();
        MySqlCommand cmd = new MySqlCommand("SELECT * from usuario " +
                                            " where IdUsu=@IdUsu", conexao);
        cmd.Parameters.AddWithValue("@IdUsu", Id);

        MySqlDataAdapter da = new MySqlDataAdapter(cmd);
        MySqlDataReader dr;

        Usuario usuario = new Usuario();
        dr = cmd.ExecuteReader(CommandBehavior.CloseConnection);
        while (dr.Read())
        {
            usuario.IdUsu = Convert.ToInt32(dr["IdUsu"]);
            usuario.nomeUsu = (string)(dr["nomeUsu"]);
            usuario.Cargo = (string)(dr["Cargo"]);
            usuario.DataNasc = Convert.ToDateTime(dr["DataNasc"]);
        }
        return usuario;
    }
```

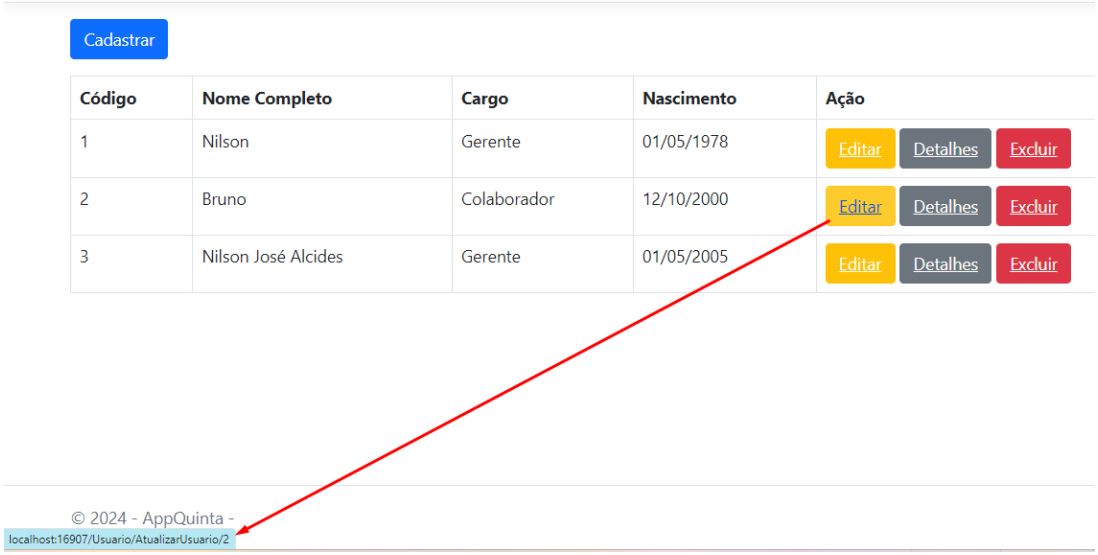
ExecuteReader() – executa um SELECT e retorna um bloco de Registros

Não podemos nos esquecer de fechar o DataReader “dr”.

Read – método para realizar a leitura de um set de dados “DataSet” “DataReader”

Precisamos ajustar os links no Index

```
@foreach (Usuario usuario in Model)
{
    <tr>
        <td>@usuario.IdUsu</td>
        <td>@usuario.nomeUsu </td>
        <td>@usuario.Cargo</td>
        <td>@usuario.DataNasc.ToString("dd/MM/yyyy")</td>
        <td>

            <button type="button" class="btn btn-warning">
                @Html.ActionLink("Editar", "AtualizarUsuario", new { id = usuario.IdUsu })
            </button>

            <button type="button" class="btn btn-secondary">
                @Html.ActionLink("Detalhes", "DetalhesUsuario", new { id = usuario.IdUsu })
            </button>

            <button type="button" class="btn btn-danger">
                @Html.ActionLink("Excluir", "ExcluirUsuario", new { id = usuario.IdUsu},
                        new { onclick = "return confirm('Deseja realmente desativar?');" })
            </button>
        </td>
    </tr>
}
```



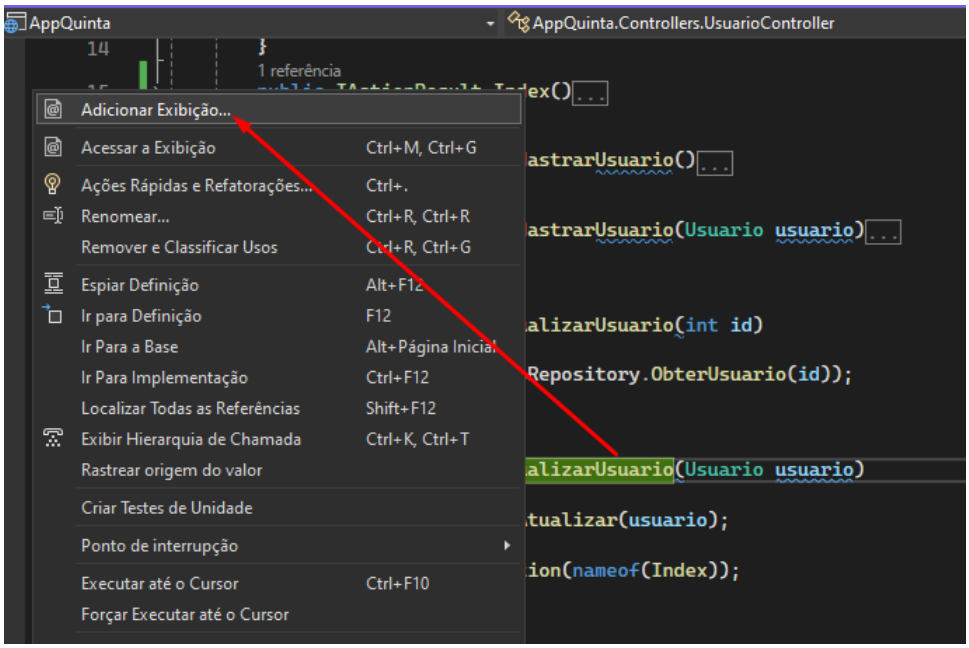
Vamos reescrever o método AtualizarUsuario dentro da classe UsuarioRepository

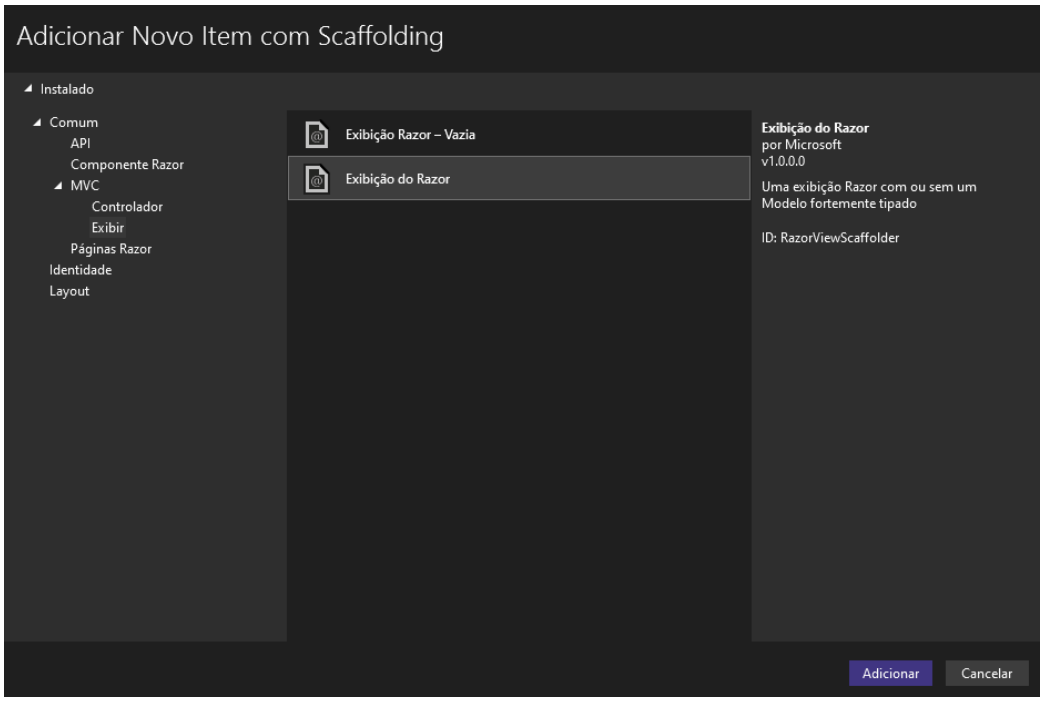
```
2 referências
public void Atualizar(Usuario usuario)
{
    using (var conexao = new MySqlConnection(_conexaoMySQL))
    {
        conexao.Open();
        MySqlCommand cmd = new MySqlCommand("Update usuario set nomeUsu=@nomeUsu, Cargo=@Cargo, " +
                                            " DataNasc=@DataNasc Where IdUsu=@IdUsu;", conexao);

        cmd.Parameters.Add("@nomeUsu", MySqlDbType.VarChar).Value = usuario.nomeUsu;
        cmd.Parameters.Add("@Cargo", MySqlDbType.VarChar).Value = usuario.Cargo;
        cmd.Parameters.Add("@DataNasc", MySqlDbType.VarChar).Value = usuario.DataNasc.ToString("yyyy/MM/dd");
        cmd.Parameters.Add("@IdUsu", MySqlDbType.VarChar).Value = usuario.IdUsu;

        cmd.ExecuteNonQuery();
        conexao.Close();
    }
}
1 referência
```

Agora vamos criar a Action para editar o usuário

```
[HttpGet]
0 referências
public IActionResult AtualizarUsuario(int id)
{
    return View(_usuarioRepository.ObterUsuario(id));
}
[HttpPost]
0 referências
public IActionResult AtualizarUsuario(Usuario usuario)
{
    _usuarioRepository.Atualizar(usuario);

    return RedirectToAction(nameof(Index));
}
```

Agora podemos criar a View


AppQuinta
AppQuinta.Controllers.UsuarioController
1 referência
Adicionar Exibição...
Acessar a Exibição Ctrl+M, Ctrl+G
Ações Rápidas e Refatorações... Ctrl+.
Renomear... Ctrl+R, Ctrl+R
Remover e Classificar Usos Ctrl+R, Ctrl+G
Espiar Definição Alt+F12
Ir para Definição F12
Ir Para a Base Alt+Página Inicial
Ir Para Implementação Ctrl+F12
Localizar Todas as Referências Shift+F12
Exibir Hierarquia de Chamada Ctrl+K, Ctrl+T
Rastrear origem do valor
Criar Testes de Unidade
Ponto de interrupção
Executar até o Cursor Ctrl+F10
Forçar Executar até o Cursor


Adicionar Novo Item com Scaffolding
Instalado
Comum
API
Componente Razor
MVC
Controlador
Exibir
Páginas Razor
Identidade
Layout
Exibição Razor – Vazia
Exibição do Razor
Exibição do Razor
por Microsoft
v1.0.0.0
Uma exibição Razor com ou sem um Modelo fortemente tipado
ID: RazorViewScaffolder
Adicionar
Cancelar

Adicionar Exibição do Razor

Nome da exibição: AtualizarUsuario

Modelo: Edit

Classe do modelo: Usuario (AppQuinta.Models)

Opções

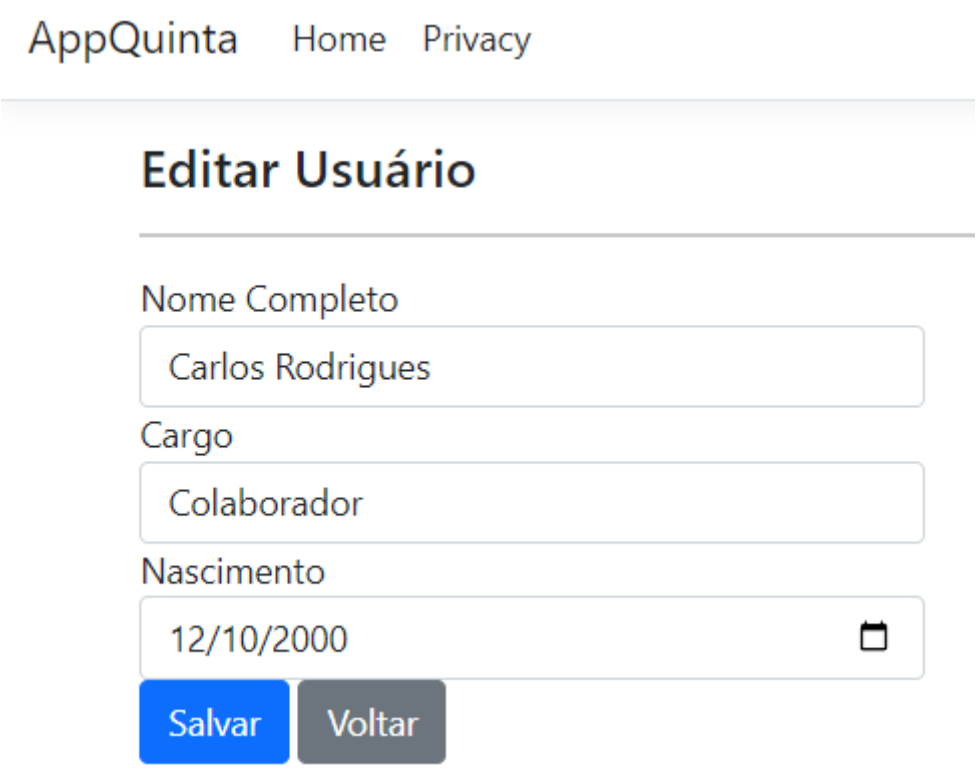
- [ ] Criar como um modo de exibição parcial
- [x] Bibliotecas de scripts de referência
- [x] Usar uma página de layout

...

(Deixe em branco se ele estiver definido em um arquivo Razor _viewstart)

Adicionar | Cancelar

```razor
@model AppQuinta.Models.Usuario

@{    ViewData["Title"] = "AtualizarUsuario"; }

<h4>Editar Usuário</h4> <hr />
<div class="row">
    <div class="col-md-4">
        <form asp-action="AtualizarUsuario">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
              <input type="hidden" asp-for="IdUsu" class="form-control" />

            <div class="form-group">
                <label asp-for="nomeUsu" class="control-label"></label>
                <input asp-for="nomeUsu" class="form-control" />
                <span asp-validation-for="nomeUsu" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Cargo" class="control-label"></label>
                <input asp-for="Cargo" class="form-control" />
                <span asp-validation-for="Cargo" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="DataNasc" class="control-label"></label>
                <input type="date" asp-for="DataNasc" class="form-control" />
                <span asp-validation-for="DataNasc" class="text-danger"></span>
            </div>
            <div class="form-group">
                <input type="submit" value="Salvar" class="btn btn-primary" />
                <a class="btn btn-secondary" asp-action="Index">Voltar</a>
            </div>
        </form>
    </div>
</div>
@section Scripts { @{await Html.RenderPartialAsync("_ValidationScriptsPartial");} }
```



Vamos reescrever o método ExcluirUsuario dentro da classe UsuarioRepository

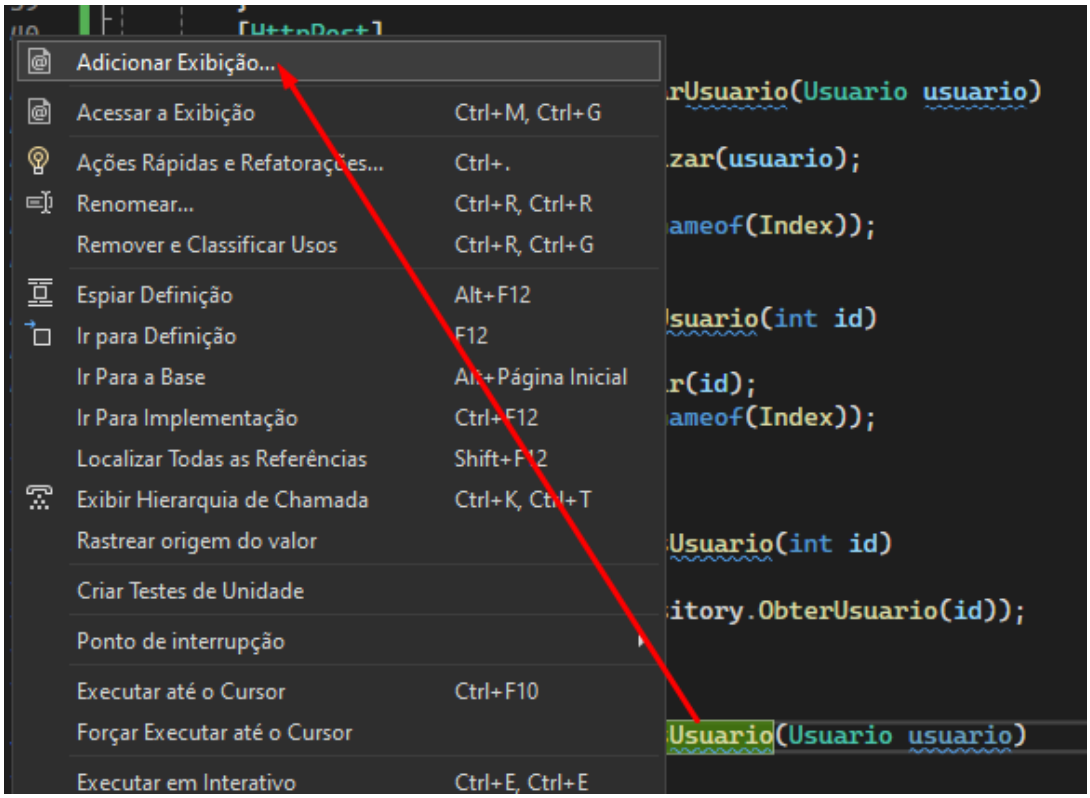
```
public void Excluir(int Id)
{
    using (var conexao = new MySqlConnection(_conexaoMySQL))
    {
        conexao.Open();
        MySqlCommand cmd = new MySqlCommand("delete from usuario where IdUsu=@IdUsu", conexao);
        cmd.Parameters.AddWithValue("@IdUsu", Id);
        int i = cmd.ExecuteNonQuery();
        conexao.Close();
    }
}
```

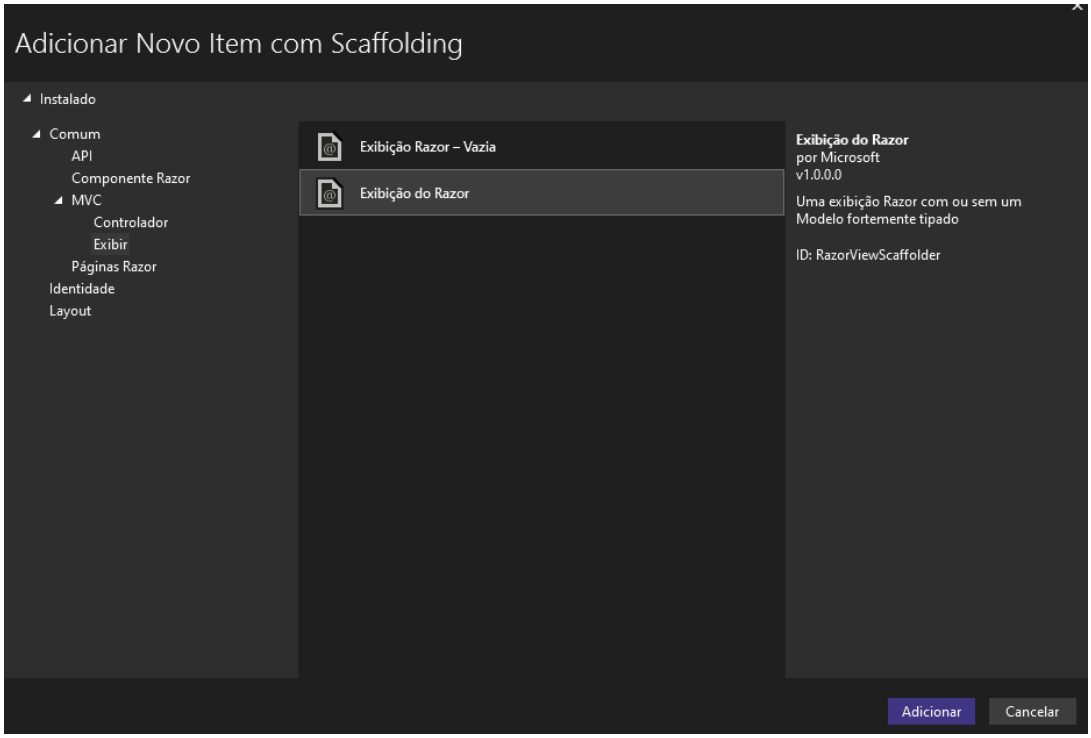
Para excluir o usuário não precisamos criar uma View.

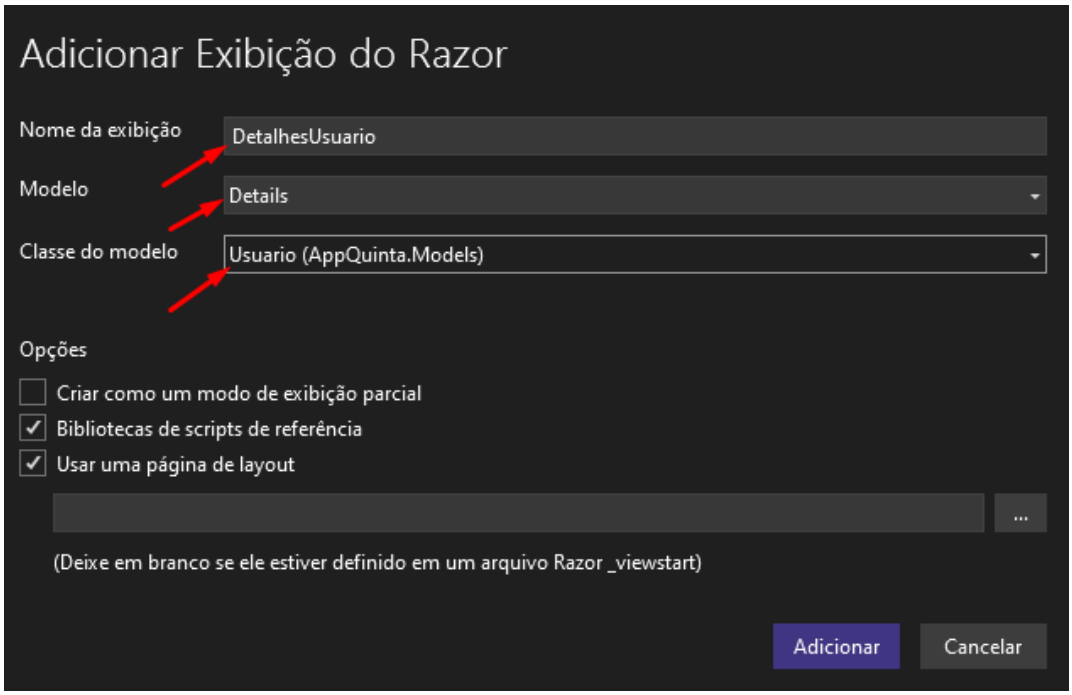
Precisamos criar o Action ExcluirUsuario

```
public IActionResult ExcluirUsuario(int id)
{
    _usuarioRepository.Excluir(id);
    return RedirectToAction(nameof(Index));
}
```

Precisamos agora criar o Action DetalhesUsuario


Adicionar Exibição...
Acessar a Exibição Ctrl+M, Ctrl+G
Ações Rápidas e Refatorações... Ctrl+.
Renomear... Ctrl+R, Ctrl+R
Remover e Classificar Usos Ctrl+R, Ctrl+G
Espiar Definição Alt+F12
Ir para Definição F12
Ir Para a Base Alt+Página Inicial
Ir Para Implementação Ctrl+F12
Localizar Todas as Referências Shift+F12
Exibir Hierarquia de Chamada Ctrl+K, Ctrl+T
Rastrear origem do valor
Criar Testes de Unidade
Ponto de interrupção
Executar até o Cursor Ctrl+F10
Forçar Executar até o Cursor
Executar em Interativo Ctrl+E, Ctrl+E


Adicionar Novo Item com Scaffolding
Instalado
Comum
API
Componente Razor
MVC
Controlador
Exibir
Páginas Razor
Identidade
Layout
Exibição Razor – Vazia
Exibição do Razor
Exibição do Razor
por Microsoft
v1.0.0.0
Uma exibição Razor com ou sem um Modelo fortemente tipado
ID: RazorViewScaffolder
Adicionar
Cancelar




```
@model AppQuinta.Models.Usuario

@{    ViewData["Title"] = "DetalhesUsuario"; }

<h1>Detalhes do Usuario</h1> <hr />

<div>
    <dl class="row">
        <dt class = "col-sm-2">@Html.DisplayNameFor(model => model.IdUsu) </dt>
        <dd class = "col-sm-10">@Html.DisplayFor(model => model.IdUsu) </dd>

        <dt class = "col-sm-2">@Html.DisplayNameFor(model => model.nomeUsu) </dt>
        <dd class = "col-sm-10">@Html.DisplayFor(model => model.nomeUsu) </dd>

        <dt class = "col-sm-2">@Html.DisplayNameFor(model => model.Cargo)</dt>
        <dd class = "col-sm-10">@Html.DisplayFor(model => model.Cargo)</dd>

        <dt class = "col-sm-2">@Html.DisplayNameFor(model => model.DataNasc)</dt>
        <dd class = "col-sm-10">@Html.DisplayFor(model => model.DataNasc)</dd>
    </dl>
</div>
<div>
    <button type="button" class="btn btn-warning">
        @Html.ActionLink("Editar", "AtualizarUsuario", new { id = Model.IdUsu })
    </button>
    <button type="button" class="btn btn-secondary">
        @Html.ActionLink("Voltar", "Index")
    </button>
</div>
```